CSE358

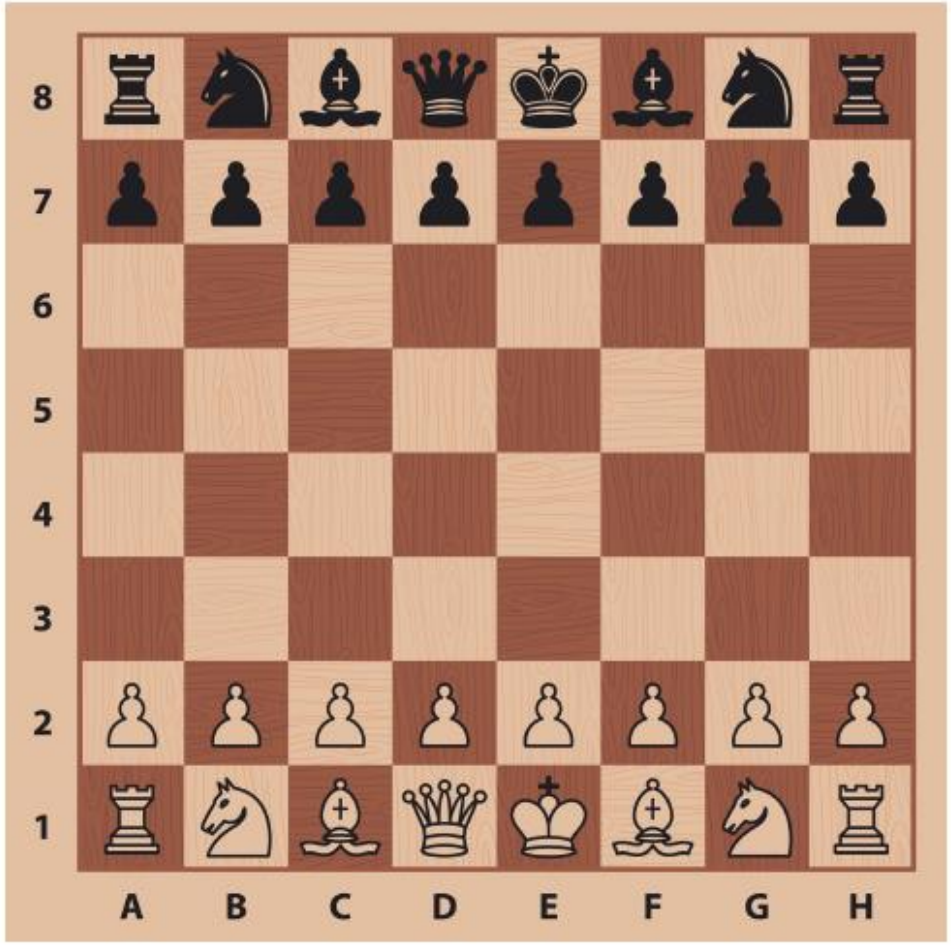
ARTIFICIAL INTELLIGENCE

05. Adversarial Search & Games

Planning ahead in a world where other agents are planning against us.

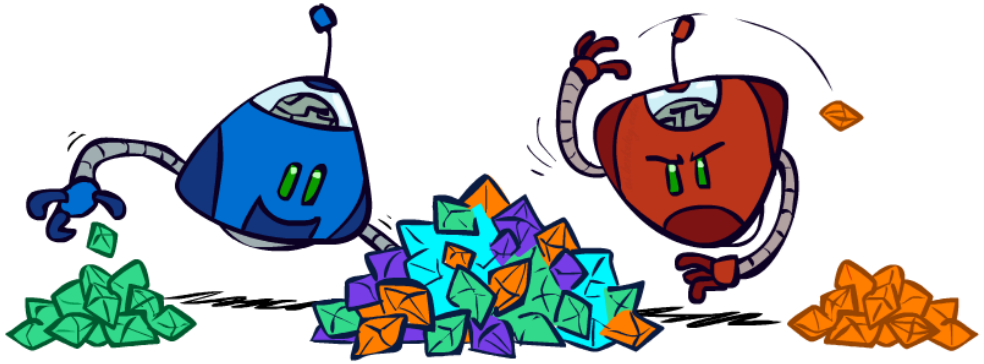
Outline

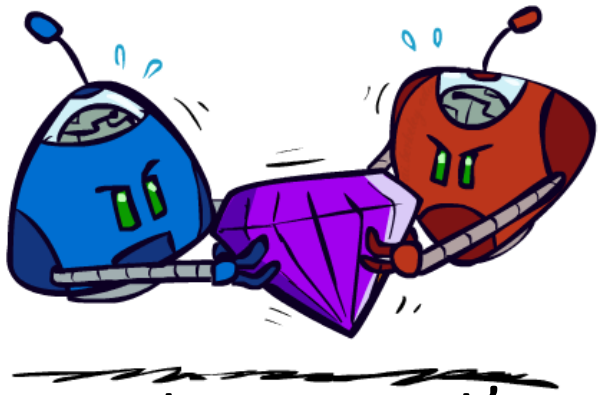
- Adversarial Search
- Minimax Algorithm
- Alpha-Beta Pruning
- Stochastic Games



Multiagent Environments

- Multiagent environments come in two versions:
  - ***cooperative*** and ***competitive***





- While problem solving in a multiagent environment, an agent's actions may depend on other agent's actions

- Typically, we are uncertain about the actions of other agents
  - But, we can often infer what actions they might take

Adversarial Search

- **Adversarial Search Problems ≡ Games**
- They occur in ***competitive*** multi-agent environments
  - There is an **opponent** we can't control planning against us!
  - Agent's goals are in conflict!! Each agent tries to optimize a ***utility***
  - ***Zero-sum-game***: One agent's gain in utility, is balanced by the opponent's loss in utility
- We will assume that there are only two agents, the environment is deterministic, and fully observable

Adversarial Search

- ***Game vs. Search:***
  - In games, we look for the optimal solution, which is not a sequence of actions anymore, but a **strategy** ("**policy**") that help us win the game
  - If opponent does ***a***, agent does ***b***, else if opponent does ***c***, agent does ***d***, etc.
- This can be expressed as a set of rules (i.e., implemented with rules)
  - However, tedious and fragile if hard-coded
- ***<u>Good news:</u>***

  Games are modeled as (1) **search problems, algorithms** and

  (2) use **heuristic evaluation functions**

  two main ingredients to model and solve games in AI

Games: *Hard Topic in AI*

- Games are a big deal in AI
- Games are interesting to AI because they are too hard to solve
- Well defined problems, clear basic solution strategies, but typically very ***huge search space***
  - Chess has a branching factor of 35
  - If each player plays 50 moves, the game tree has $35^{100}\ nodes\ \approx\ 10^{154}\ nodes$
- Main work is in figuring out how to make "good" decisions in the game, even when the optimal decision cannot be computed (infeasible)
  - Key to solve games in AI → "***Approximation***"

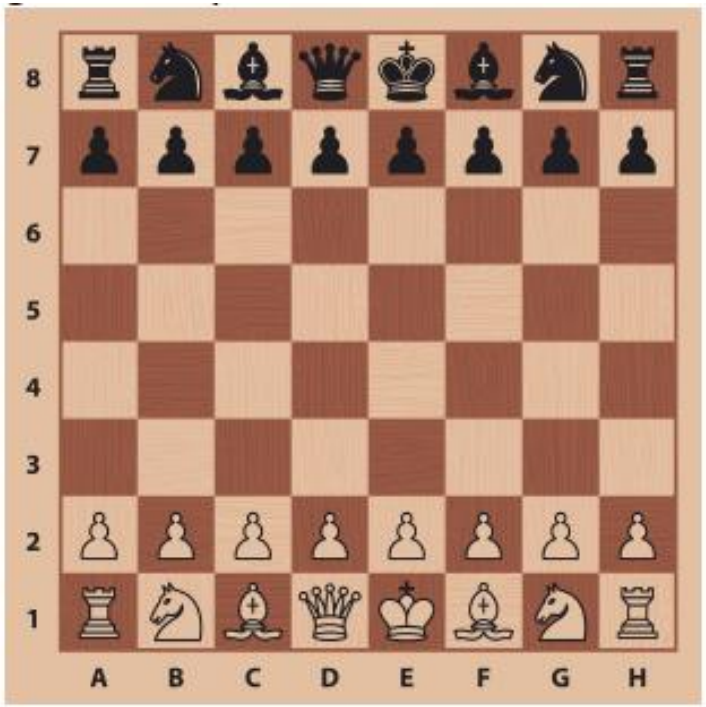
Adversarial Search

Checkers

- In 1994: "**Chinook**" ended 40-year-reign of human world champion Marion Tinsley
  - World's first computer program to win a world championship against a human in checkers
  - Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 (443 billion) positions
- In 2007: Jonathan Schaeffer et al. "**Checkers Is Solved**"
  - Shows that perfect play by both players leads to a draw
  - https://spectrum.ieee.org/computing/software/checkers-solved

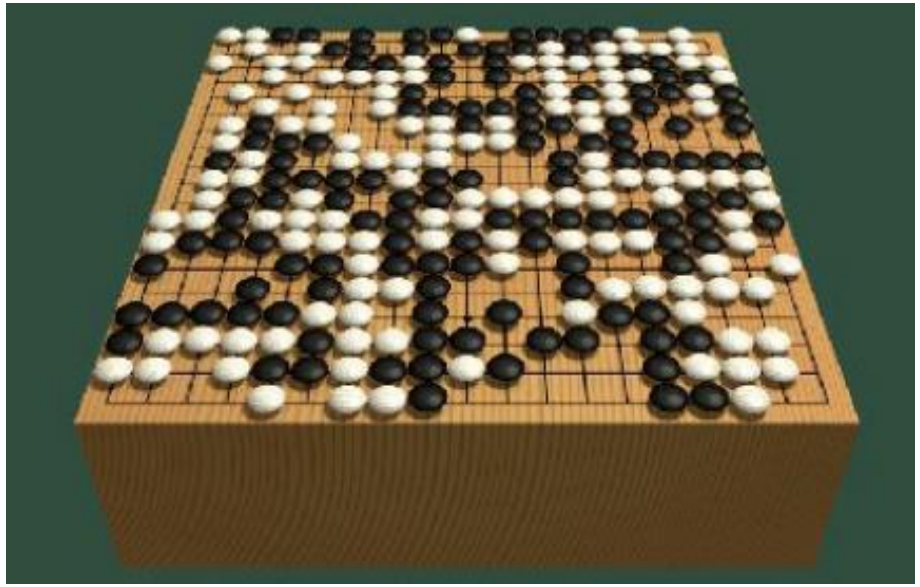
Adversarial Search

Chess



- In 1949: Caude E. Shannon in his paper "*Programming a Computer for Playing Chess*", suggested ***Chess* as an AI problem** for the community
  - Invented the **minimax algorithm** and suggested **heuristic evaluation functions** that actually help cutoff the search space
- In 1997: **Deep Blue** (the IBM intelligent machine) defeated human world champion Gary Kasparov in a six-game match
- In 2006: The undisputed world champion, Vladimir Kramnik, was defeated 4-2 by **Deep Fritz**

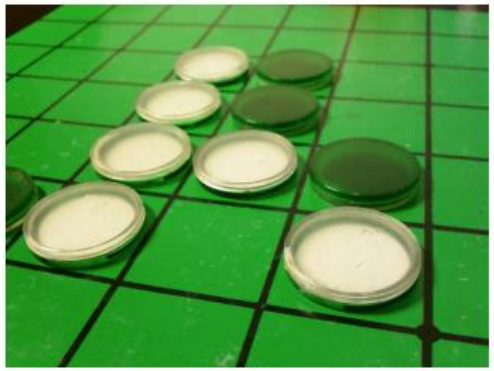
Adversarial Search



Go

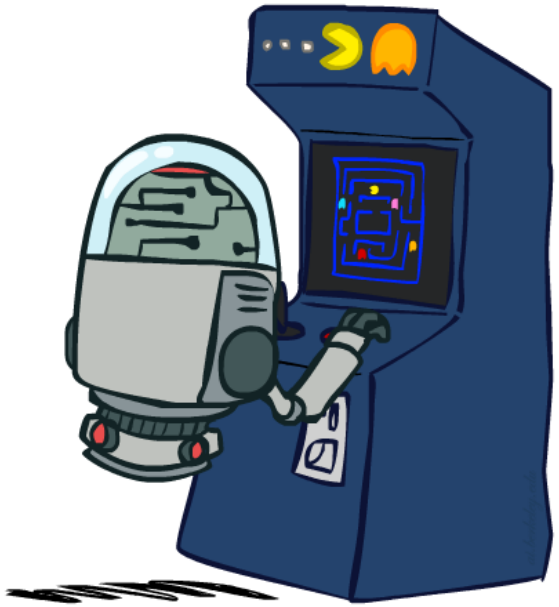
- Players can put a stone anywhere on the board

  19 x 19 = 361 possibilities for the 1st move
- b > 300!
- In 2016: Google Deep mind Project **AlphaGo** beat both Fan Hui, the European Go champion and Lee Sedol the world champion
  - Deep neural networks

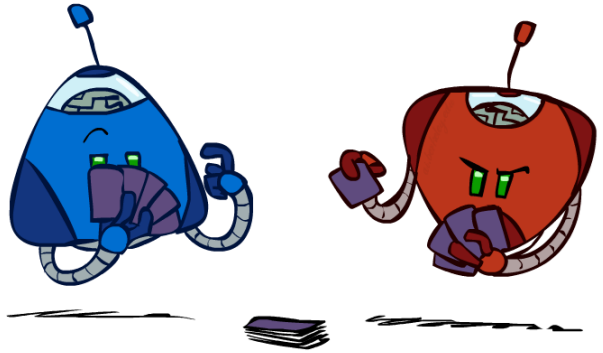


Othello / Reversi

- Several computer othello exists and human champions refuse to play against computers, that are too good (no chance to win)



Types of Games



	Deterministic	**Chance** *(Nondeterministic or Stochastic)*
Perfect Information *(Fully Observable)*	Chess, Checkers, Go, Othello	Backgammon, Monopoly
Imperfect Information *(Partially Observable)*	Battleships, Blind Tic-Tac-Toe	Bridge, Poker, Scrabble, Nuclear War

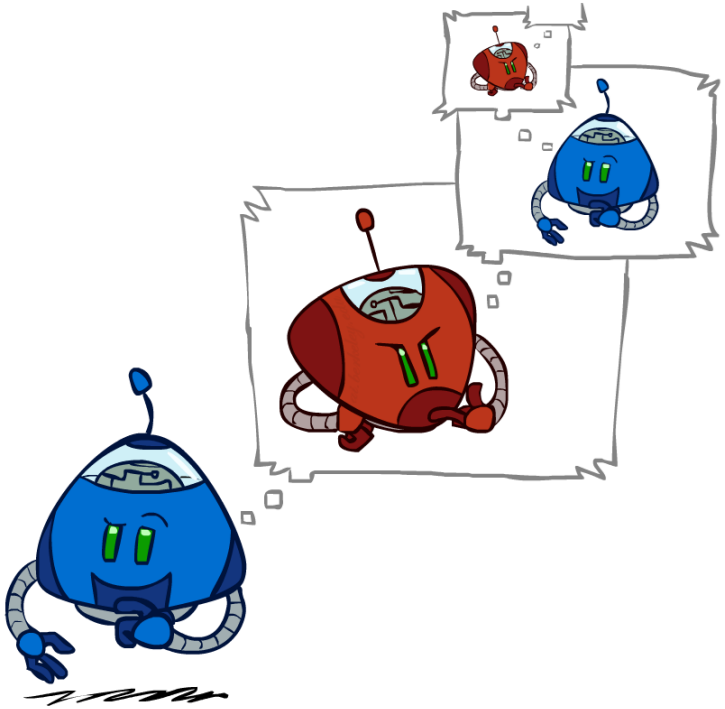
We are mostly interested in deterministic games, fully observable environments, **zero-sum**, where two agents act alternately

Zero-Sum Games

- Adversarial Games that involve pure competition
- Agents have different values on the outcomes
- One agent maximizes one single value, while the other minimizes it
- Each move in the game by one of the players is called a "***ply***"

One function: ***one agents maximizes it and one minimizes it!***

Embedded Thinking…



Embedded thinking or backward reasoning!

- **One agent is trying to figure out what to do**
- How to decide?
  - He thinks about the consequences of the possible actions
  - He needs to think about his opponent as well...
  - The opponent is also thinking about what to do etc.
  - Each will imagine what would be the response from the opponent to their actions
- This leads to ***embedded thinking***!

- Assume that the players are <u>rational</u>: They will play the move that optimizes their performance measure

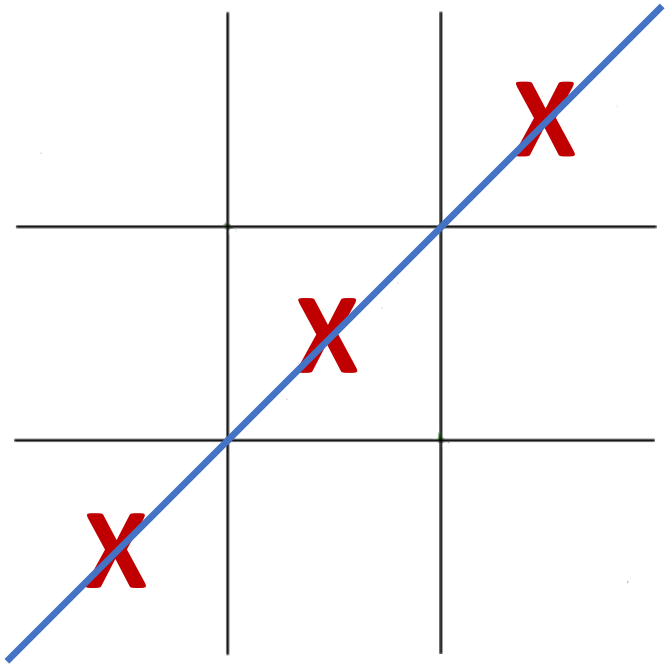
Formalization of Adversarial Search Problems

A game can be formally defined as a kind of search problem

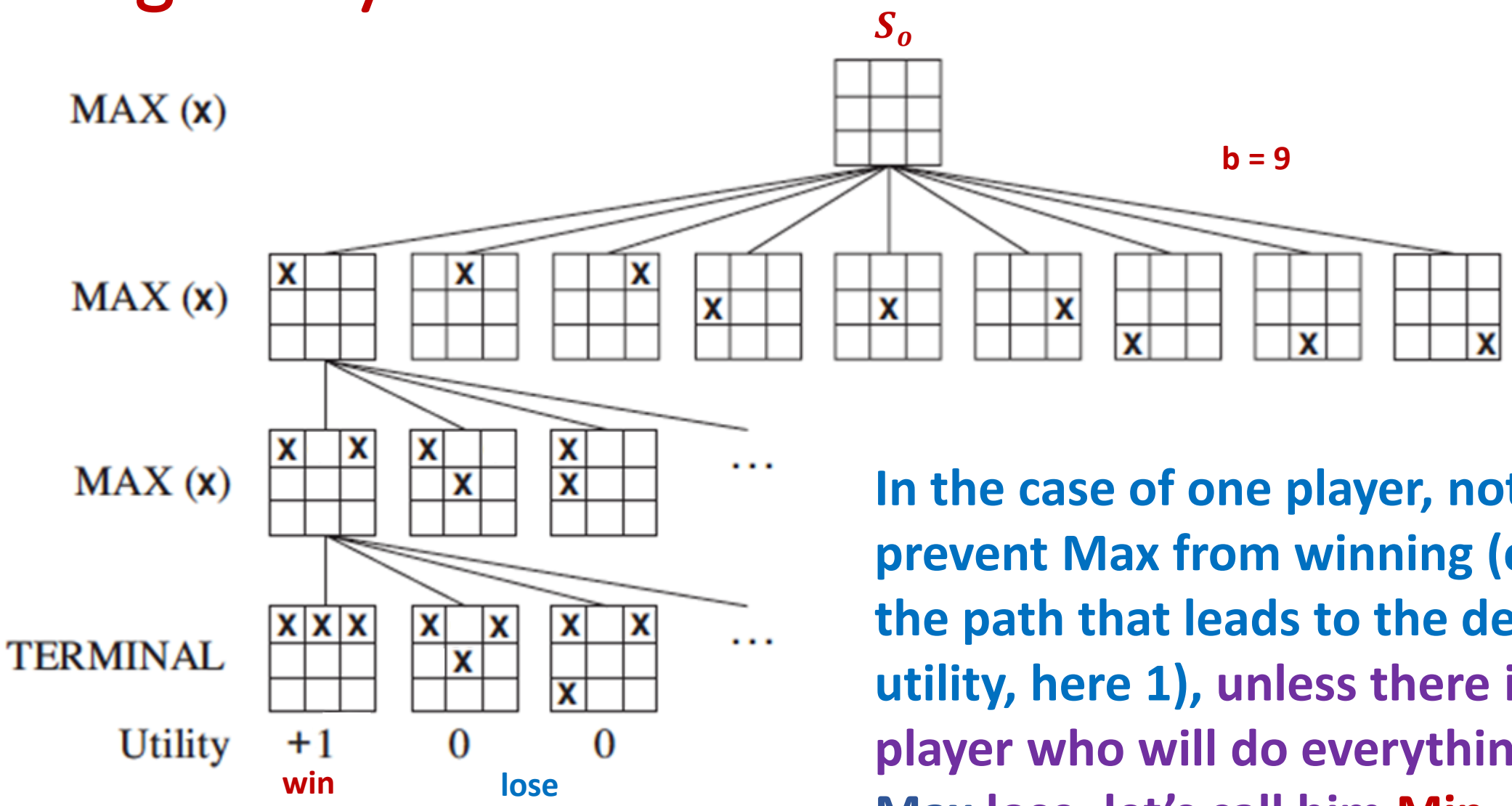
- The **initial state** - S_o - model of the state of the environment that the agent starts from
- **`Player(s)`:** Whose turn is it?
  - Defines which player has the move in state ***s***. Usually by taking turns.
- **`Actions(s)`:** returns the set of legal moves in state ***s***
- **`Result(s, a)`:** Transition model ($S \times A \rightarrow S$) which defines the result of a move
- **`Terminal test`:** Is terminal? Game over?
  - ***True*** when the game is over, ***False*** otherwise
  - States where game ends are called **terminal states**
- ***`Utility(s, p)`:*** **utility function** or **objective function** for a game that ends in terminal state ***s*** for player ***p***
  - In Chess, the outcome is a **win**, **loss**, or **draw** with values **+1**, **0**, **½**
  - For tic-tac-toe we can use a utility of **+1**, **-1**, **0**, if it's a **win**, **loss**, or **draw**, respectively

Single Player…

- Assume we have a tic-tac-toe with one player
- Let's call him ***Max*** and have him play ***three moves*** only for the sake of the example



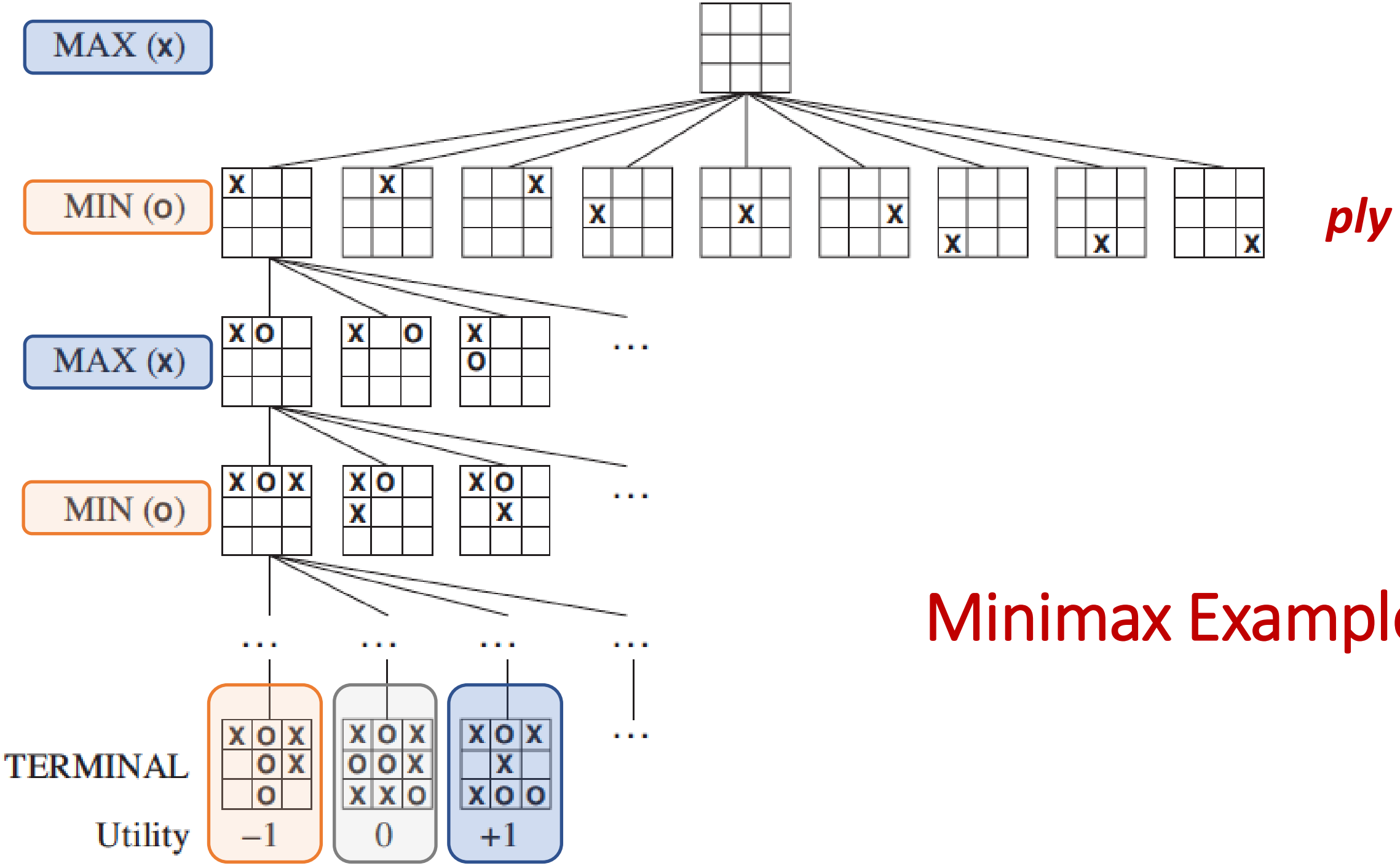
Single Player…



In the case of one player, nothing will prevent Max from winning (choose the path that leads to the desired utility, here 1), unless there is another player who will do everything to make Max lose, let's call him **Min**

Adversarial Search: Minimax

- Two players: ***Max*** and ***Min***
- Players alternate turns
- Max moves first
- Max maximizes results
- Min minimizes the result
- Compute each node's minimax value's the best achievable utility against an optimal adversary
- ***Minimax value*** $\equiv$ ***best achievable payoff against best play***



Minimax Example

Adversarial Search: Minimax

- Aims to find the optimal strategy for Max to win:
  - Depth-first search of the game tree
  - An optimal leaf node could appear at any depth of the tree
  - ***Minimax principle:*** compute the utility of being in a state, assuming both players play optimally from there until the end of the game
  - *Idea of the minimax algorithm:* Propagate **minimax** values up the tree once terminal nodes are discovered
    - Go all the way down to the tree and down to the leaves in the tree

The Minimax Algorithm

- Minimax algorithm is designed to determine the optimal strategy for MAX: *Perfect play for deterministic, perfect-information games*
- Idea: choose move to position with ***highest minimax value*** ≡ **best achievable payoff against best play**
- An optimal procedure
  1. Depth-first search of the game tree
  2. Apply utility (payoff) function to each leaf
  3. Back-up values from leaves through branch nodes:
     - A MAX node computes the max of its child values
     - A MIN node computes the min of its child values
  4. At root: Choose move leading to the child of highest value

Adversarial Search: Minimax

- If state is **terminal node**: Value is ***utility(state)***
- If state is **MAX node**: Value is ***highest value of all successor node values*** (children)
- If state is **MIN node**: Value is ***lowest value of all successor node values*** (children)
- ***Recursive Definition:***

For a state s minimax(s) =

$$\begin{cases} Utility(s) & \text{if Terminal-test(s)} \\ max_{a \in Actions(s)} \text{minimax(Result(s,a))} & \text{if Player(s) = Max} \\ min_{a \in Actions(s)} \text{minimax(Result(s,a))} & \text{if Player(s) = Min} \end{cases}$$

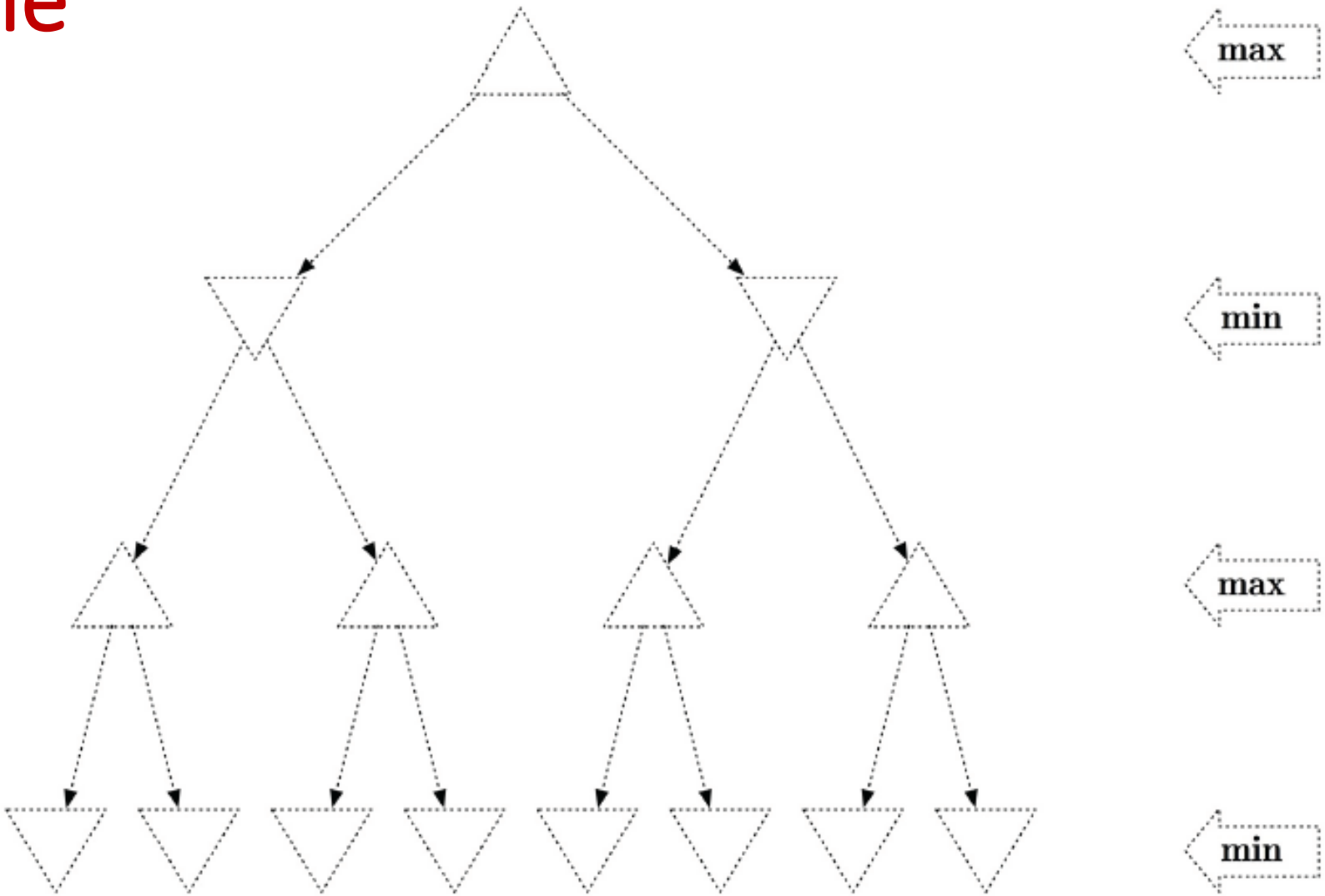
The Minimax Algorithm

```
/* Find the child state with the lowest utility value */

function MINIMIZE(state)
    returns TUPLE of ⟨STATE, UTILITY⟩ :

    if TERMINAL-TEST(state):
        return ⟨NULL, EVAL(state)⟩

    ⟨minChild, minUtility⟩ = ⟨NULL, ∞⟩

    for child in state.children():
        ⟨_, utility⟩ = MAXIMIZE(child)

        if utility < minUtility:
            ⟨minChild, minUtility⟩ = ⟨child, utility⟩

    return ⟨minChild, minUtility⟩
```

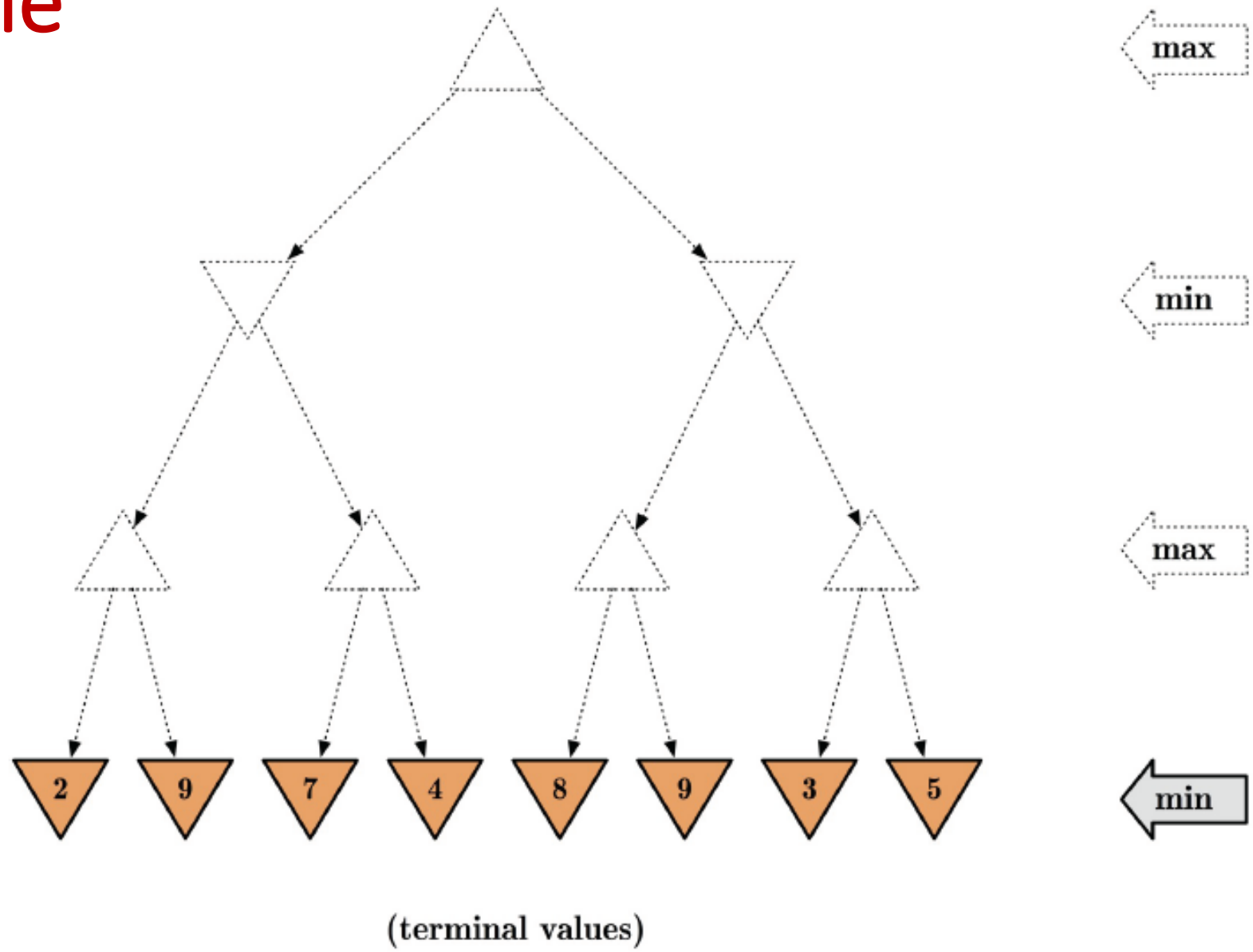
```
/* Find the child state with the highest utility value */

function MAXIMIZE(state)
    returns TUPLE of ⟨STATE, UTILITY⟩ :

    if TERMINAL-TEST(state):
        return ⟨NULL, EVAL(state)⟩

    ⟨maxChild, maxUtility⟩ = ⟨NULL, −∞⟩

    for child in state.children():
        ⟨_, utility⟩ = MINIMIZE(child)

        if utility > maxUtility:
            ⟨maxChild, maxUtility⟩ = ⟨child, utility⟩

    return ⟨maxChild, maxUtility⟩
```

```
/* Find the child state with the highest utility value */

function DECISION(state)
    returns STATE :

    ⟨child, _⟩ = MAXIMIZE(state)

    return child
```

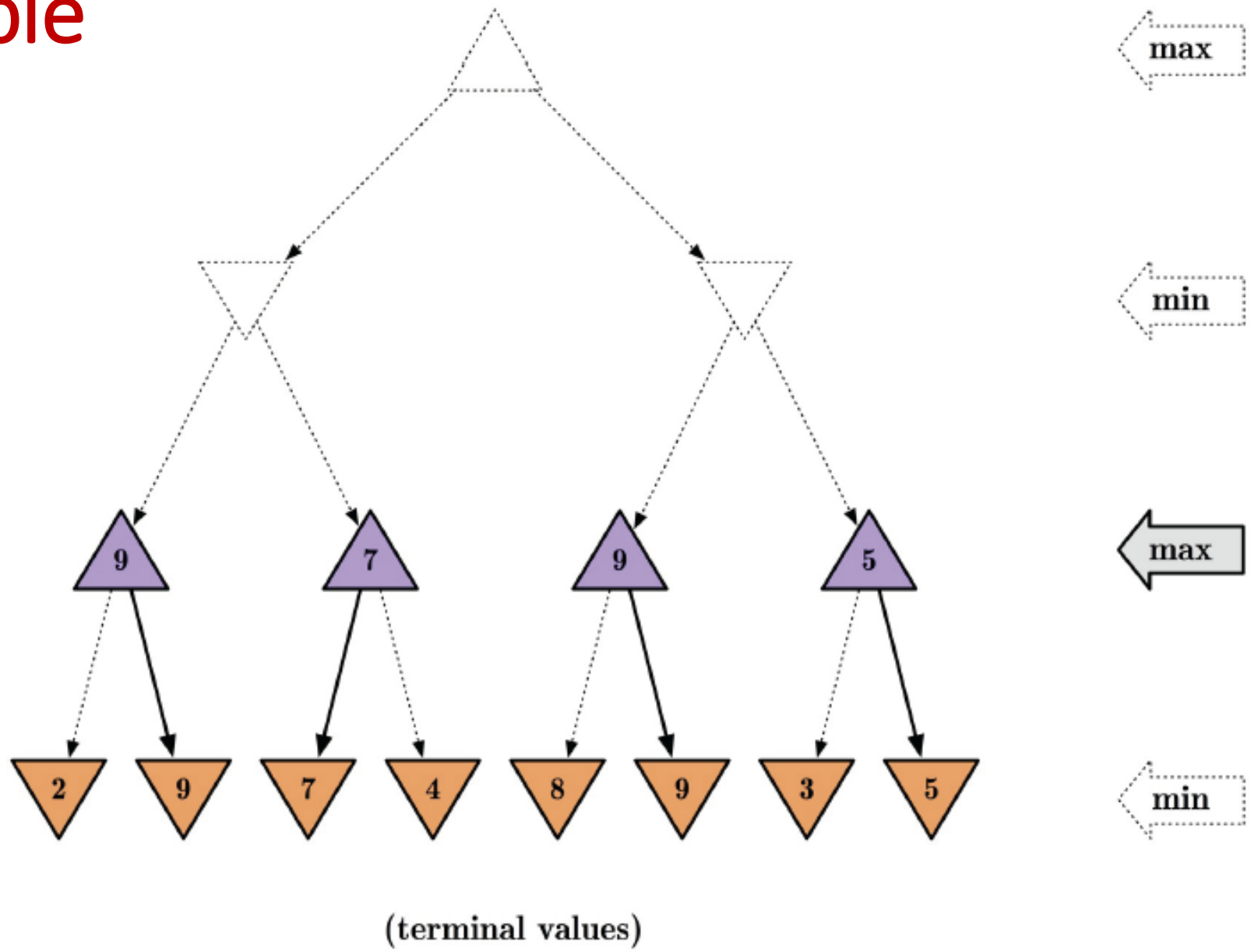
Minimax Example



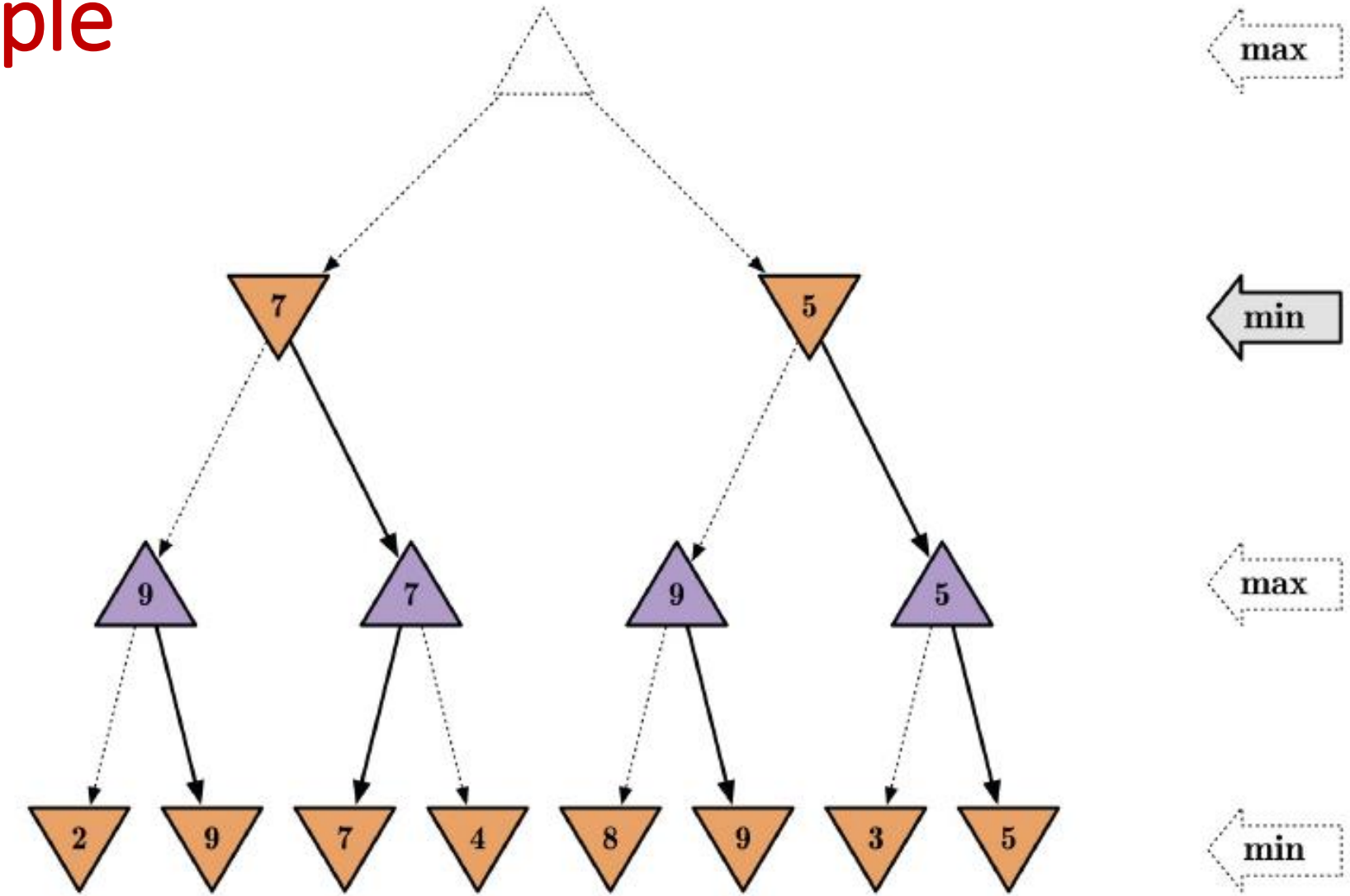
Minimax Example



Minimax Example

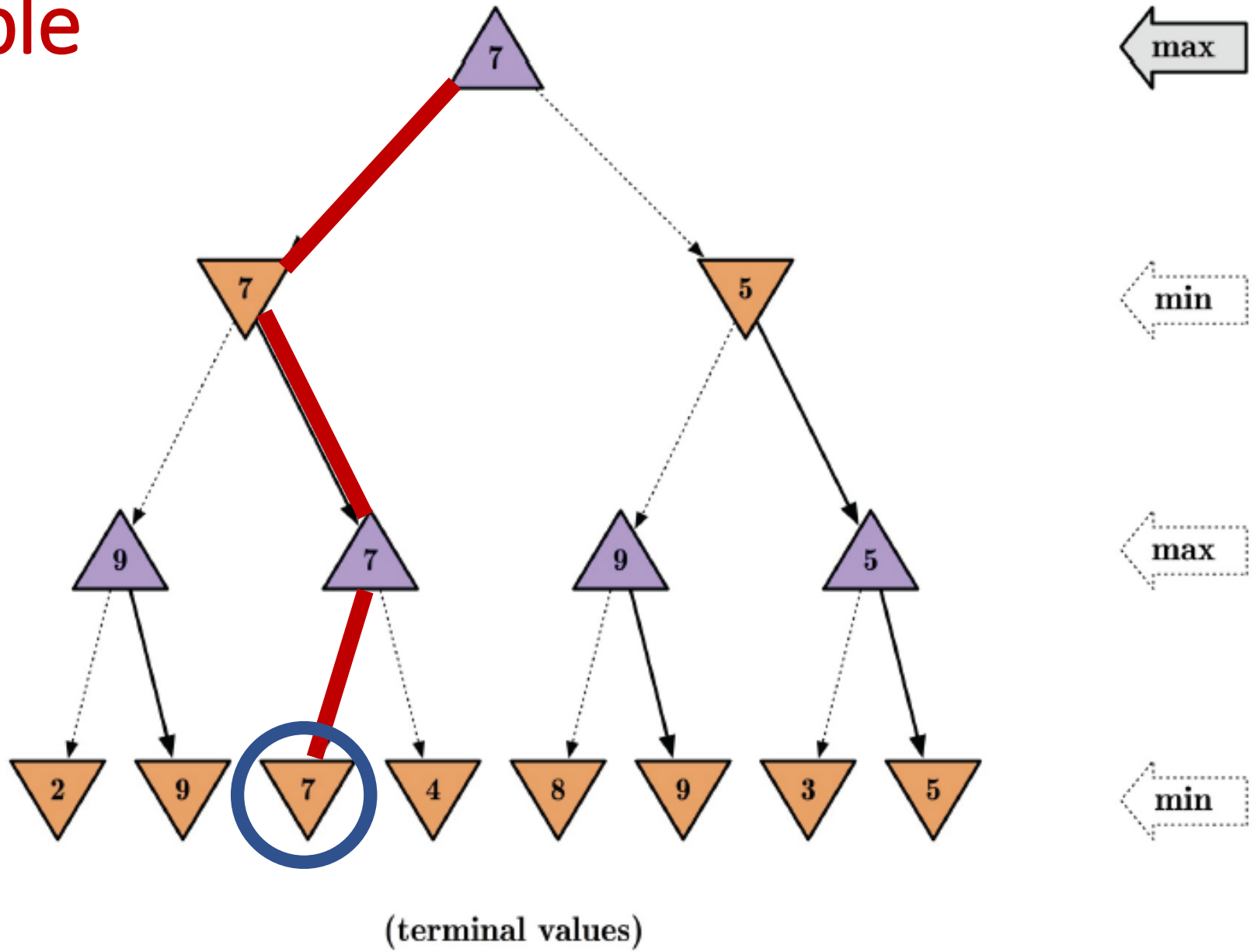


Minimax Example



(terminal values)

Minimax Example



Properties of Minimax

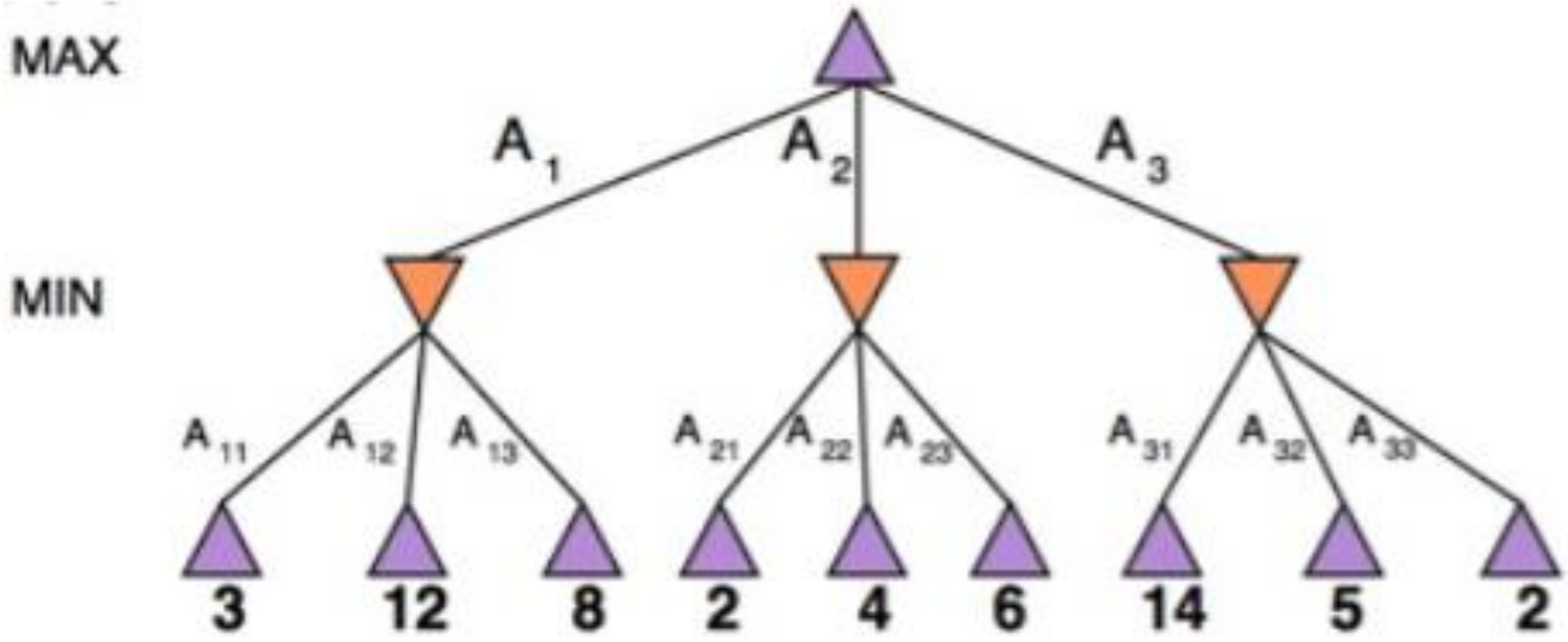
- ***Optimal*** (opponent plays optimally)
- ***Complete*** (finite search tree – possibly huge)
- ***DFS space***: $O(bm)$
- ***DFS time***: $O(b^m)$
  - **Tic-Tac-Toe**
    - * $b \approx 5$ legal moves on average, total of 9 moves (9 plies), so $m = 9$
    - * $5^9 = 1{,}953{,}125$
    - * $9! = 362{,}880$ terminal nodes
  - **Chess**
    - * $b \approx 35$ (average branching factor)
    - * $d \approx 100$ (depth of game tree for a typical game)
    - * $b^d \approx 35^{100} \approx 10^{154}$ nodes
  - **Go** branching factor starts at 361 (19×19 board)

Real Games are Hard!!

Case of Limited Resources

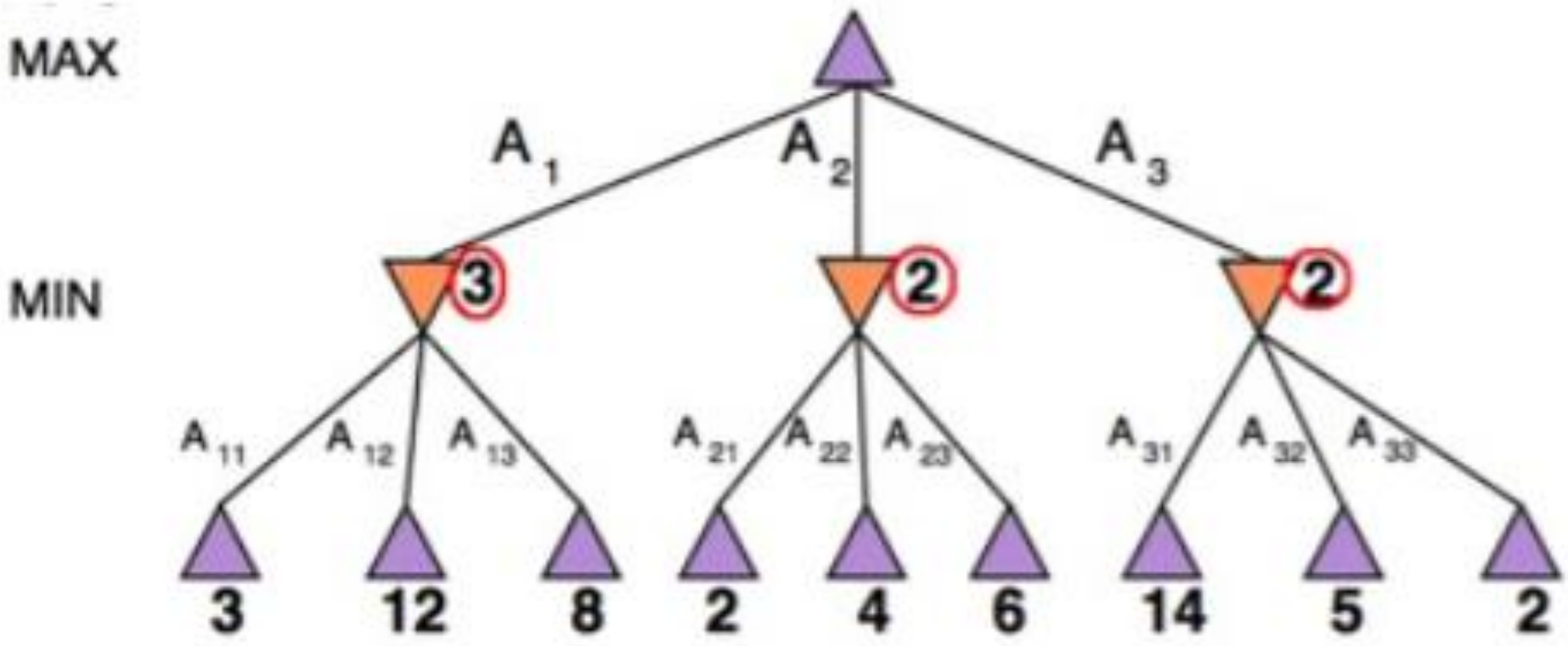
- ***Problem:*** In real games, **we are limited in time, so we can't search the leaves**
  - Complete Minimax search is infeasible
- To be practical and run in a reasonable amount of time, minimax can only search to some depth
- But, more plies make a big difference
  - You can ***plan better*** when you know more about the different consequences of those choices of actions
- ***Solution:*** *(for reducing the search space)*
  1. Replace terminal utilities with an evaluation function for non-terminal positions
  2. Use Iterative Deepening Search (IDS)
  3. Use pruning: Eliminate large parts of the tree

Alpha – Beta $(\alpha - \beta)$ pruning



A two-ply game tree

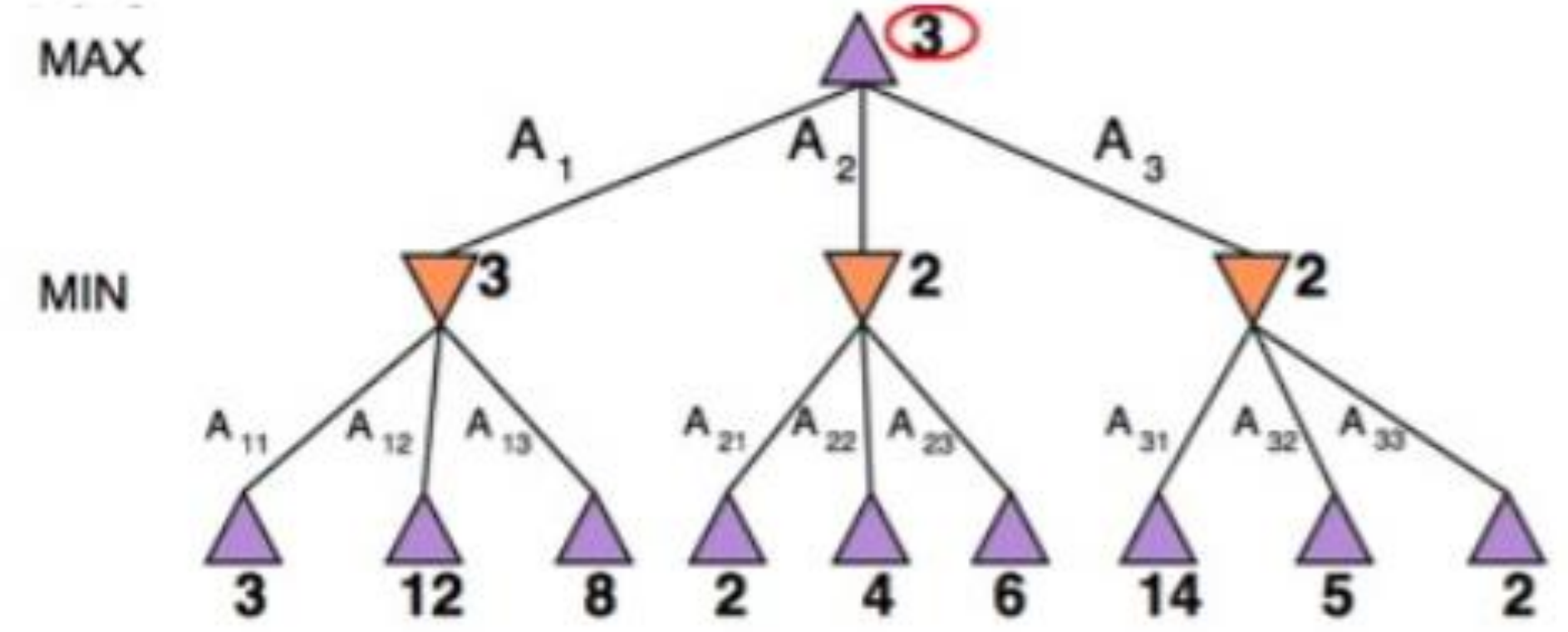
$\alpha - \beta$ pruning



A two-ply game tree

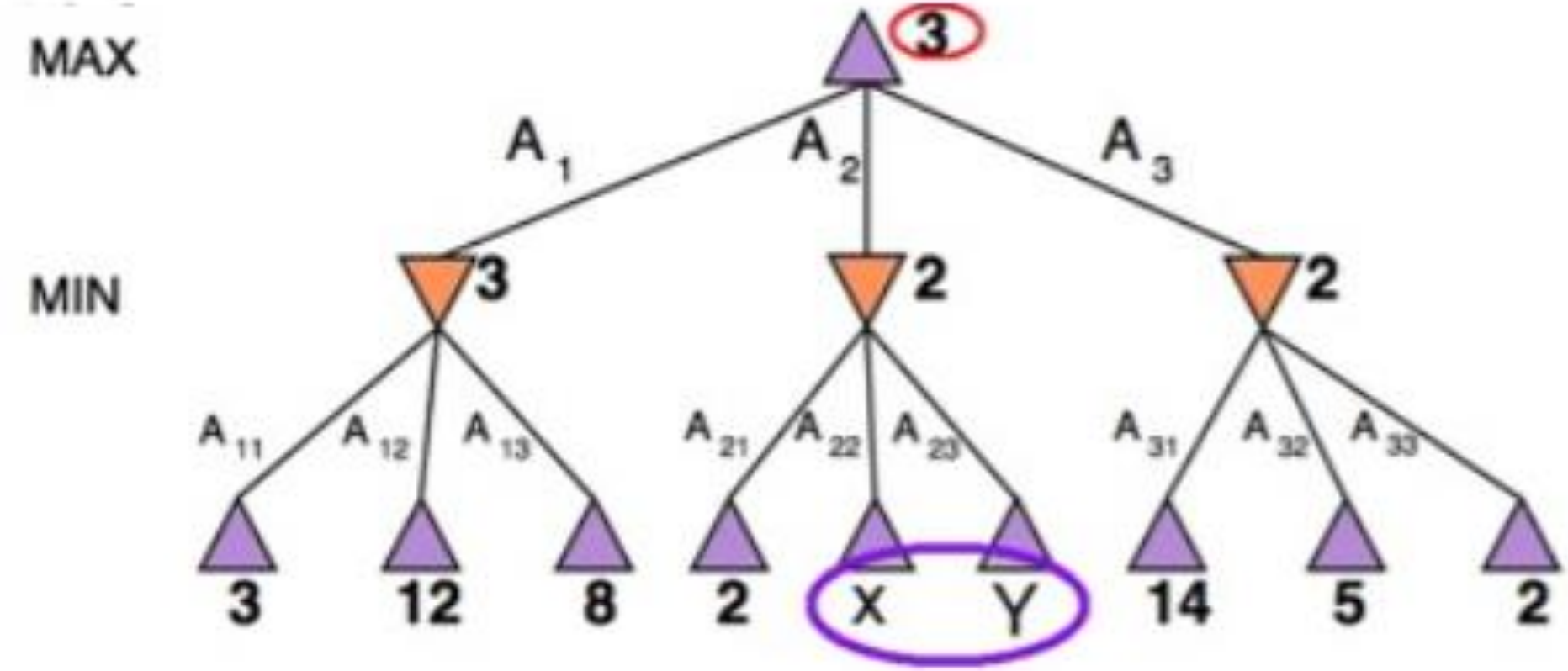
$\alpha - \beta$ pruning

Do we really need to explore the whole tree to find a minimax strategy ?



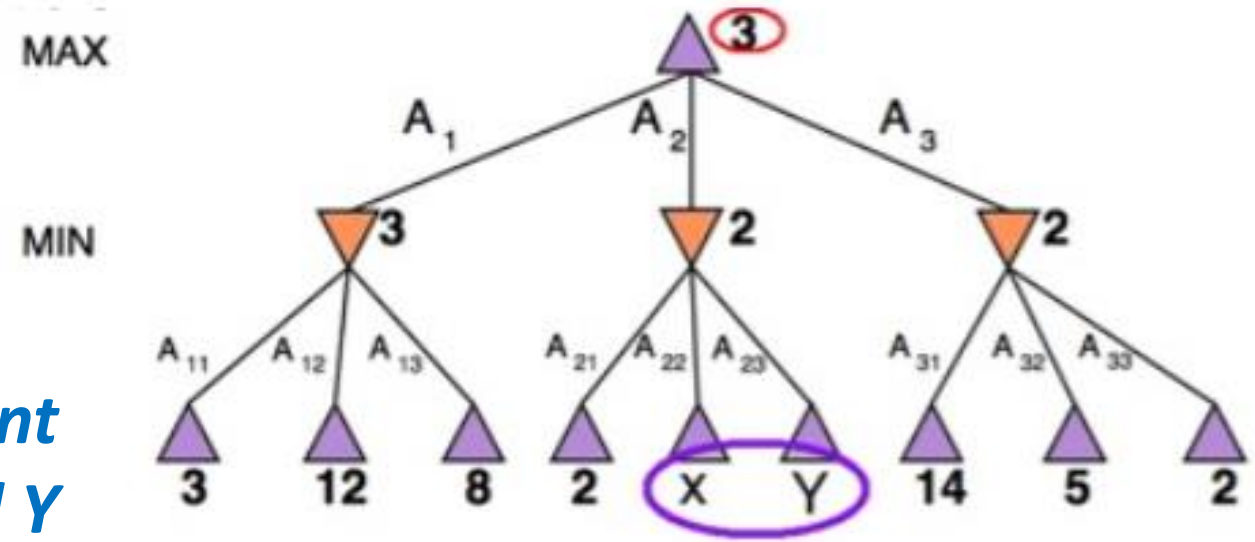
A two-ply game tree

$\alpha - \beta$ pruning



Which values are necessary?

$\alpha - \beta$ pruning



Minimax decisions are independent of the values of X and Y

$$Minimax(root) = max(min(3, 12, 8), min(2, X, Y), min(14, 5, 2))$$

$$= max(3, min(2, X, Y), 2)$$

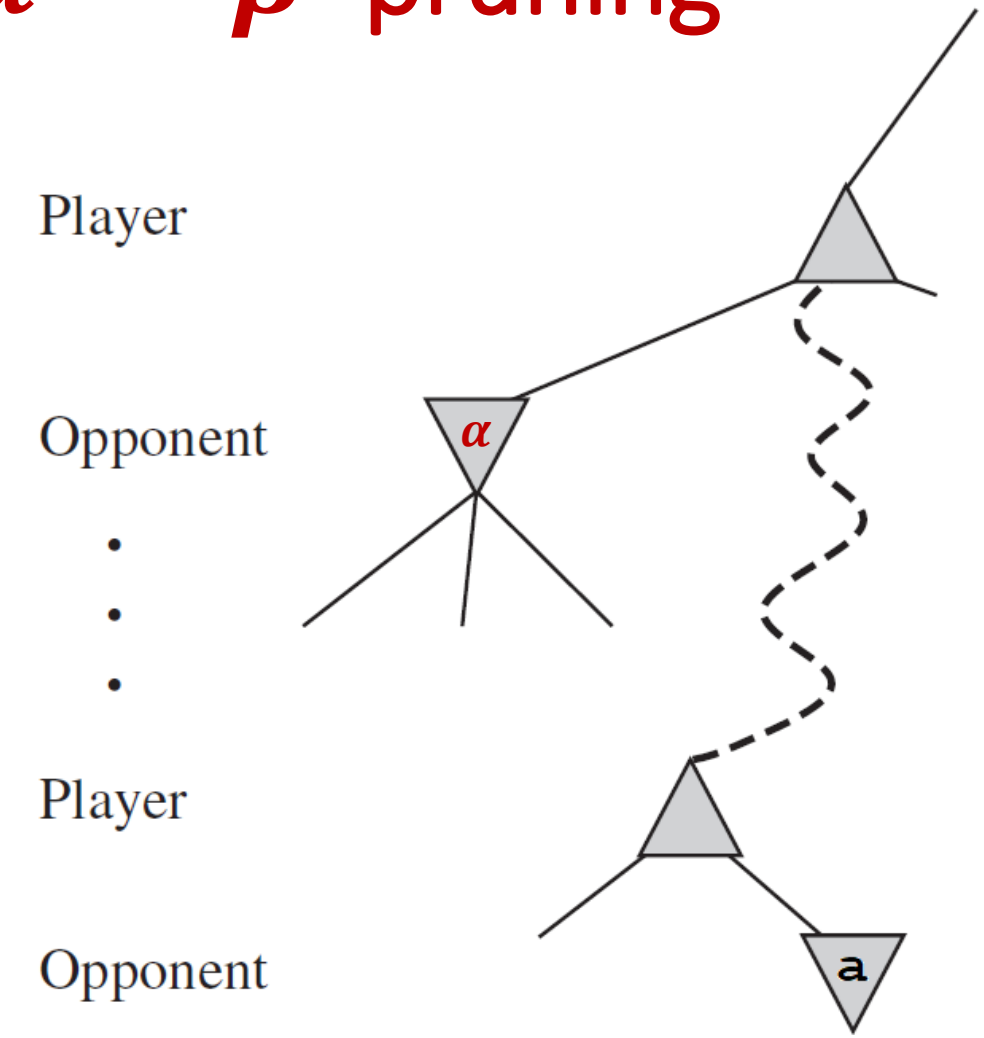
$$= max(3, Z, 2) \qquad \text{where } Z = min(2, X, Y) \leq 2$$

$$= 3$$

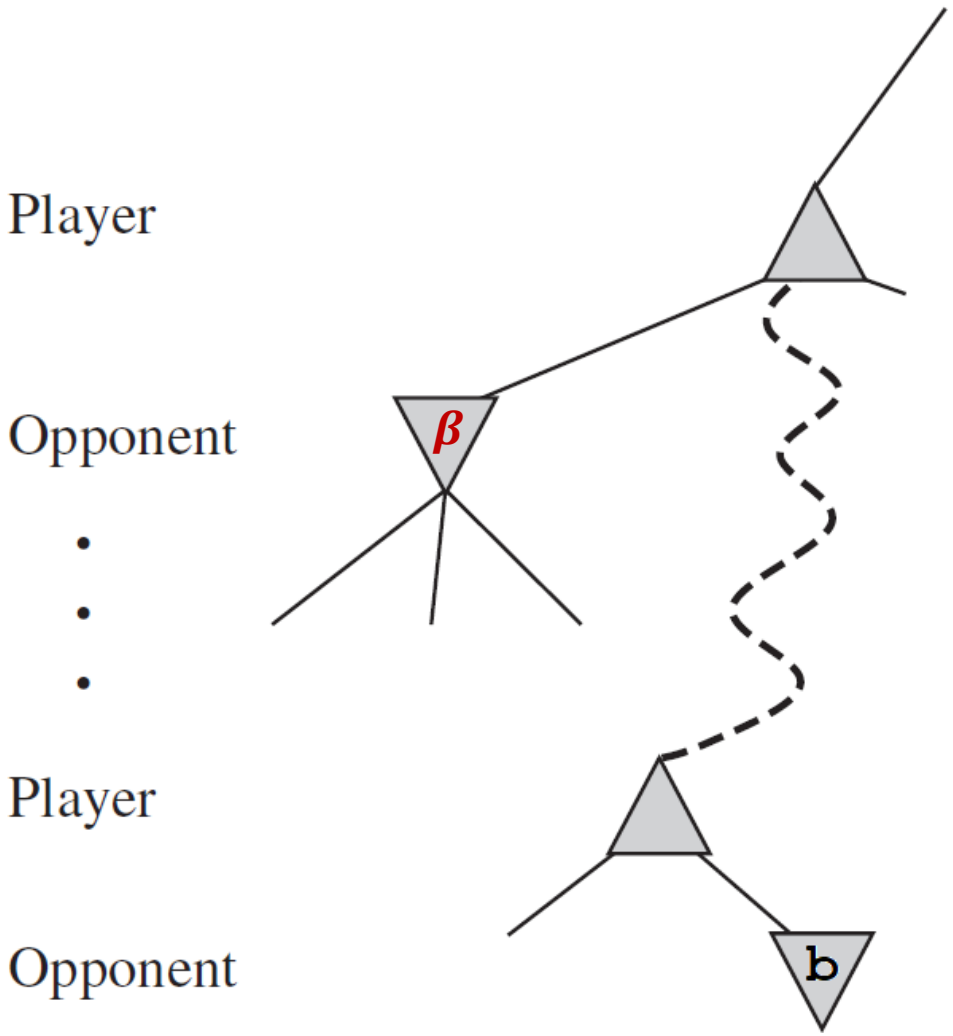
$\alpha - \beta$ pruning

- **Strategy:** Just like minimax, it performs a DFS
- **Parameters:** Keep track of two bounds
  - α : **largest** value for **MAX** across seen children (current <u>lower bound</u> on MAX's outcome)
  - β : **lowest** value for **MIN** across seen children (current <u>upper bound</u> on MIN's outcome)
- **Initialization:** $\boldsymbol{\alpha} = -\infty, \quad \boldsymbol{\beta} = \infty$
- **Propagation:** Send $\boldsymbol{\alpha}, \boldsymbol{\beta}$ values ***down*** during the search to be used for pruning
  - Update $\boldsymbol{\alpha}, \boldsymbol{\beta}$ values by ***propagating upwards*** values of terminal nodes
  - Update $\boldsymbol{\alpha}$ only at **MAX nodes** and update $\boldsymbol{\beta}$ only at **MIN nodes**
- **Pruning:** Prune any remaining branches whenever $\boldsymbol{\alpha} \geq \boldsymbol{\beta}$

$\alpha - \beta$ pruning



If α is better than **a** for **Max**, then **Max** will avoid it, that is prune that branch



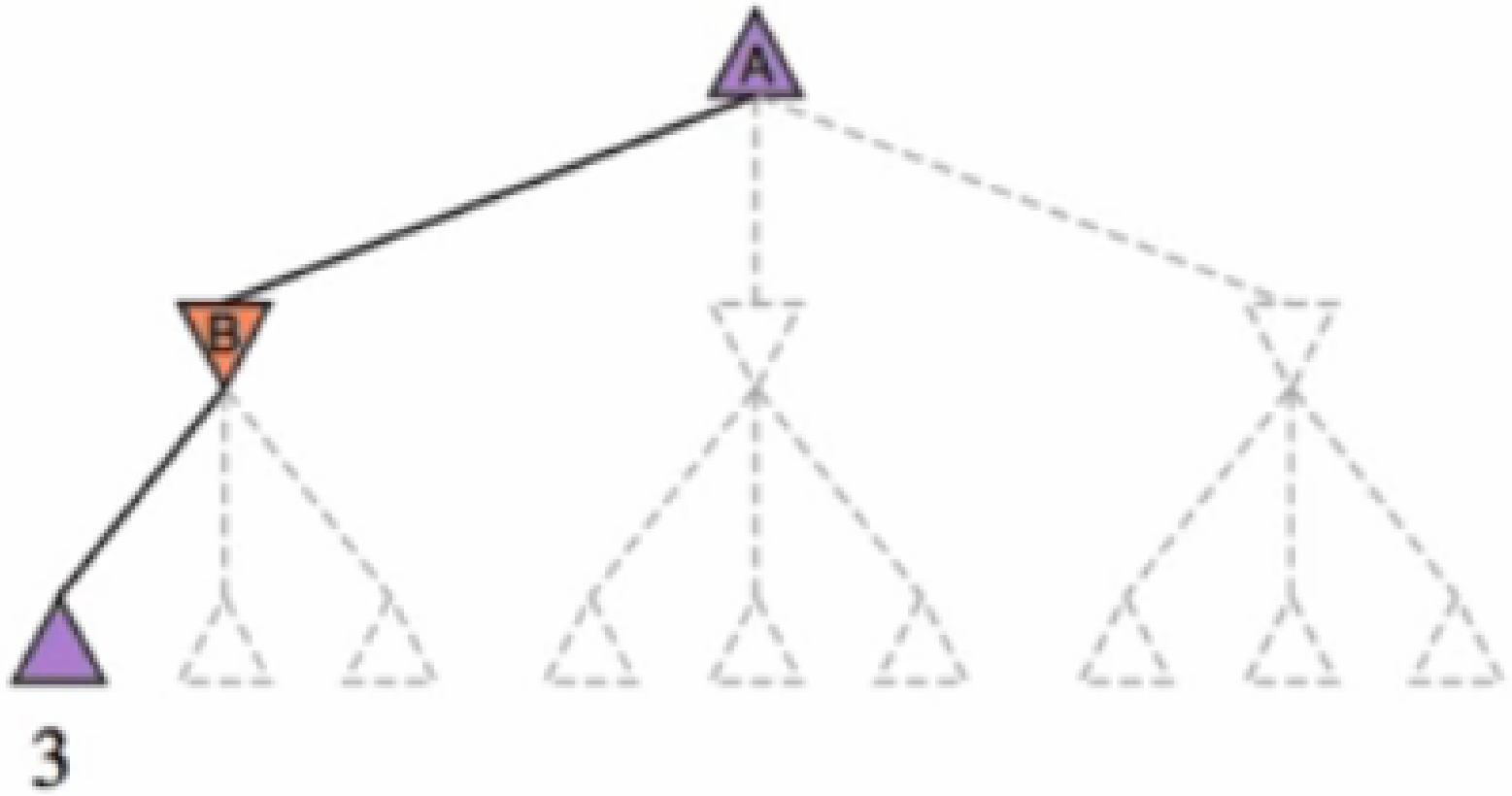
If β is better than **b** for **Min**, then **Min** will avoid it, that is prune that branch

Example: $\alpha - \beta$ pruning

MAX

MIN

(a)

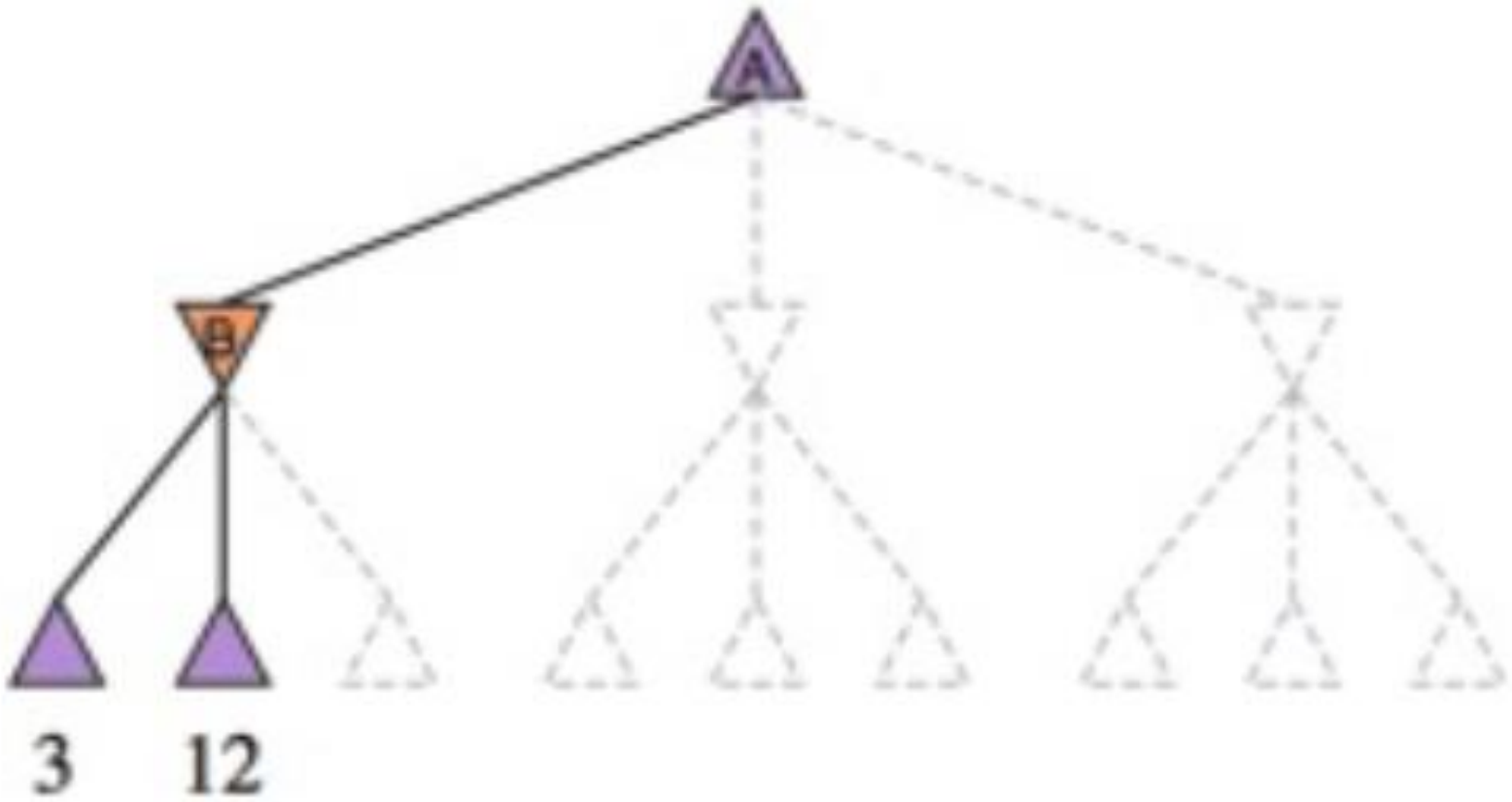


Example: $\alpha - \beta$ pruning

MAX

MIN

(b)

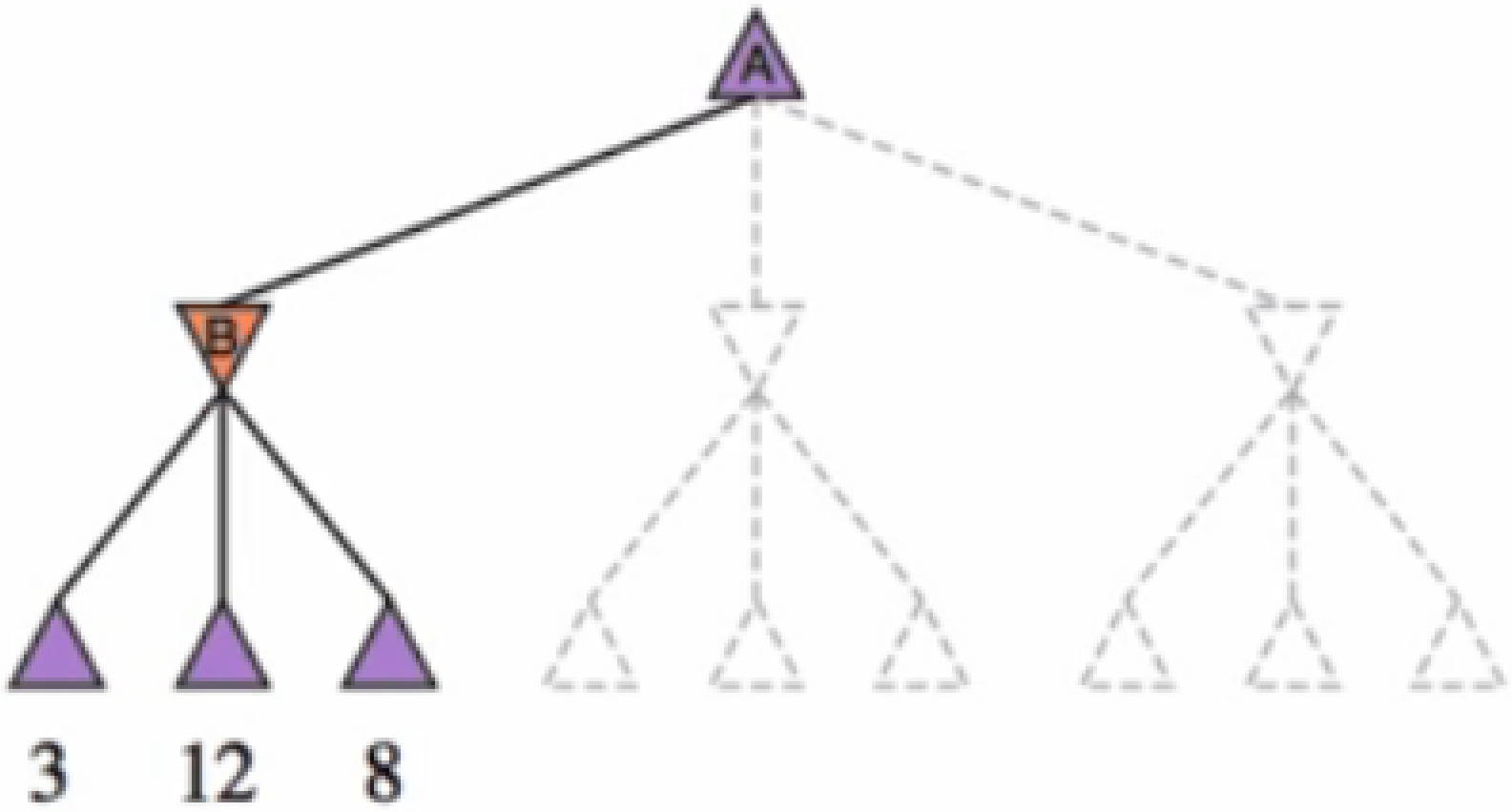


Example: $\alpha - \beta$ pruning

MAX

MIN

(c)

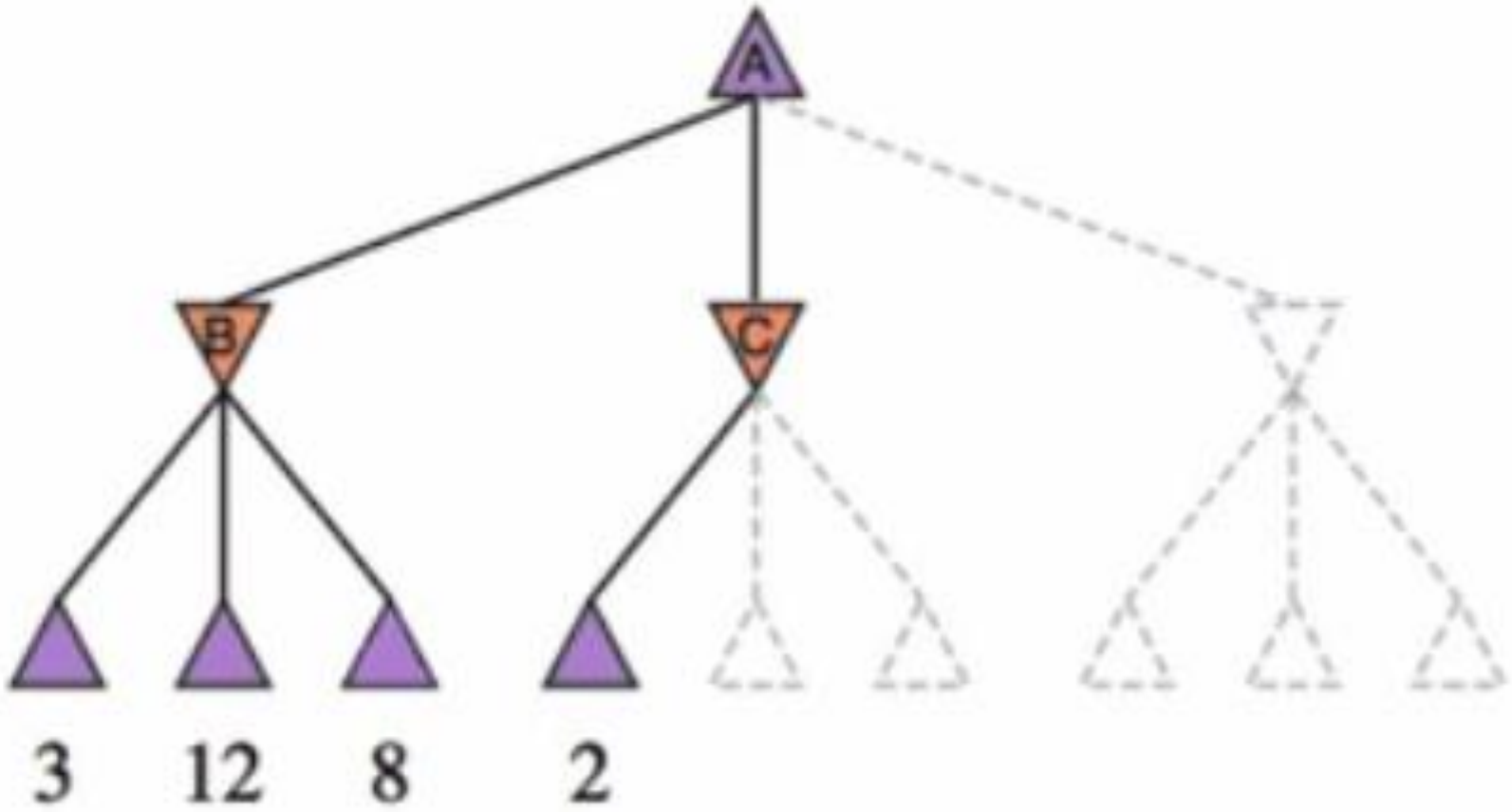


Example: $\alpha - \beta$ pruning

MAX

MIN

(d)

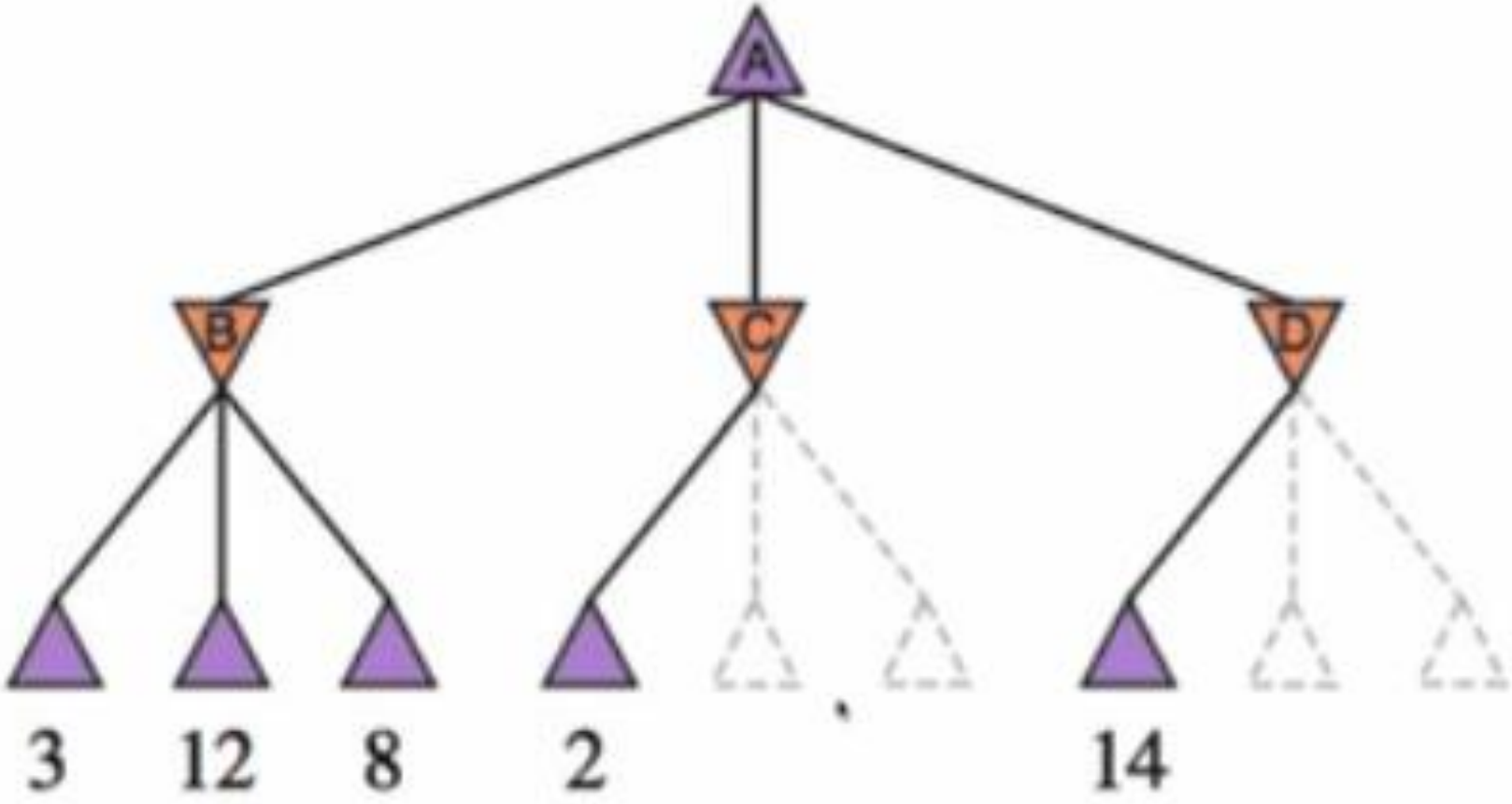


Example: $\alpha - \beta$ pruning

MAX

MIN

(e)

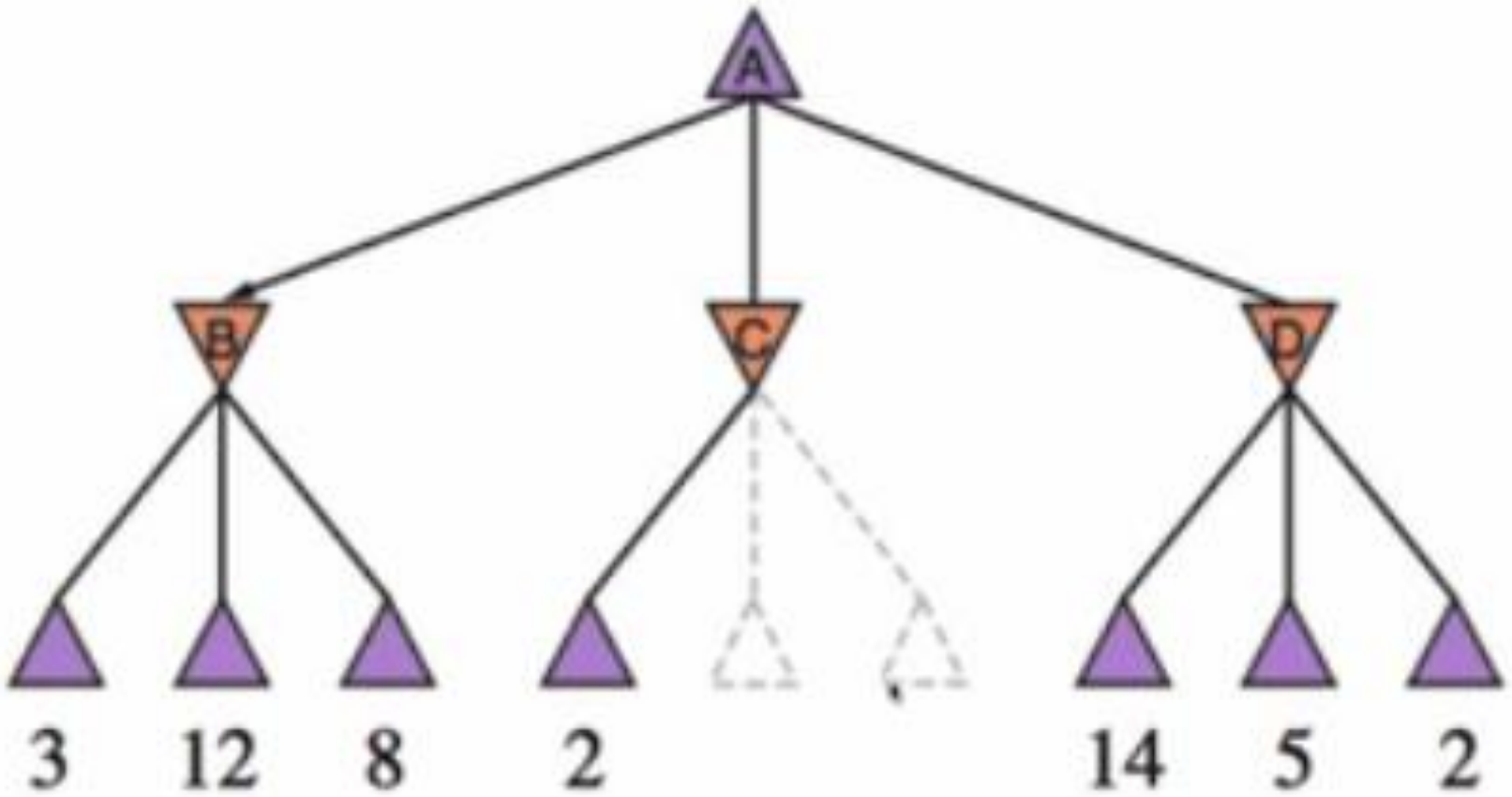


Example: $\alpha - \beta$ pruning

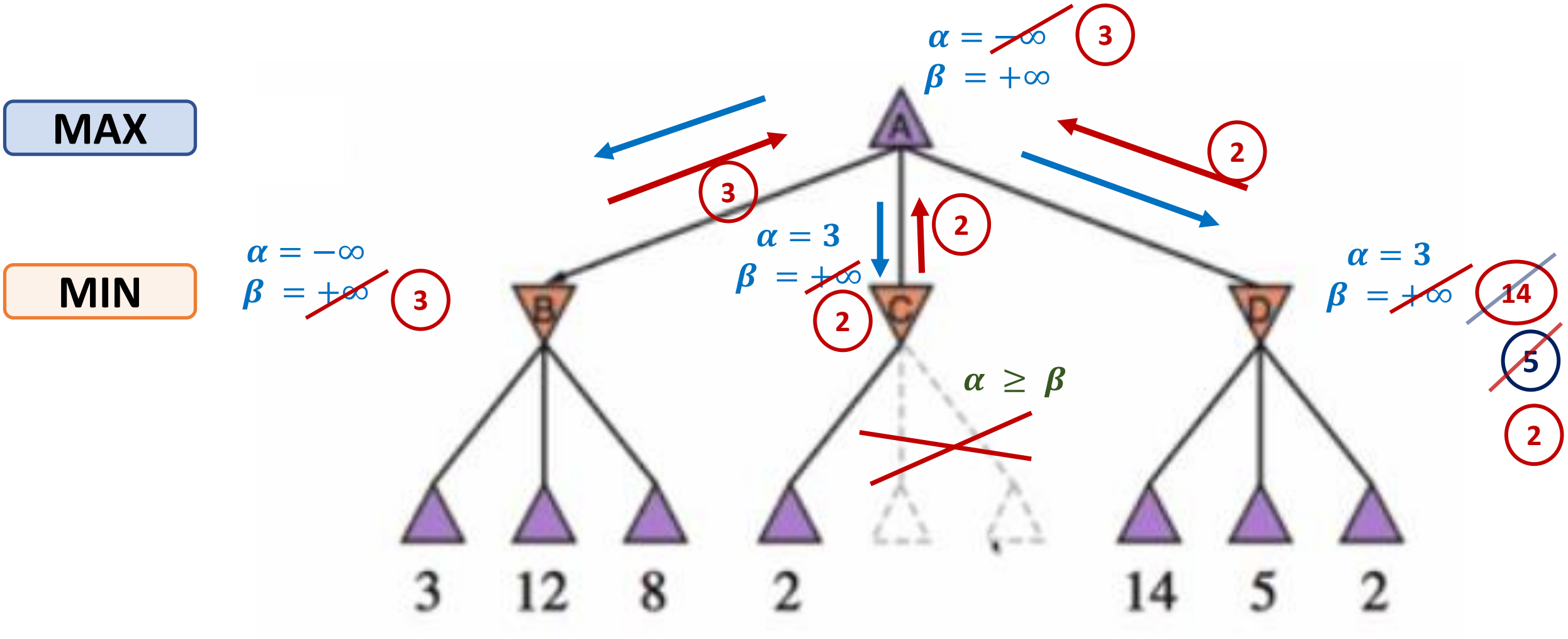
MAX

MIN

(f)



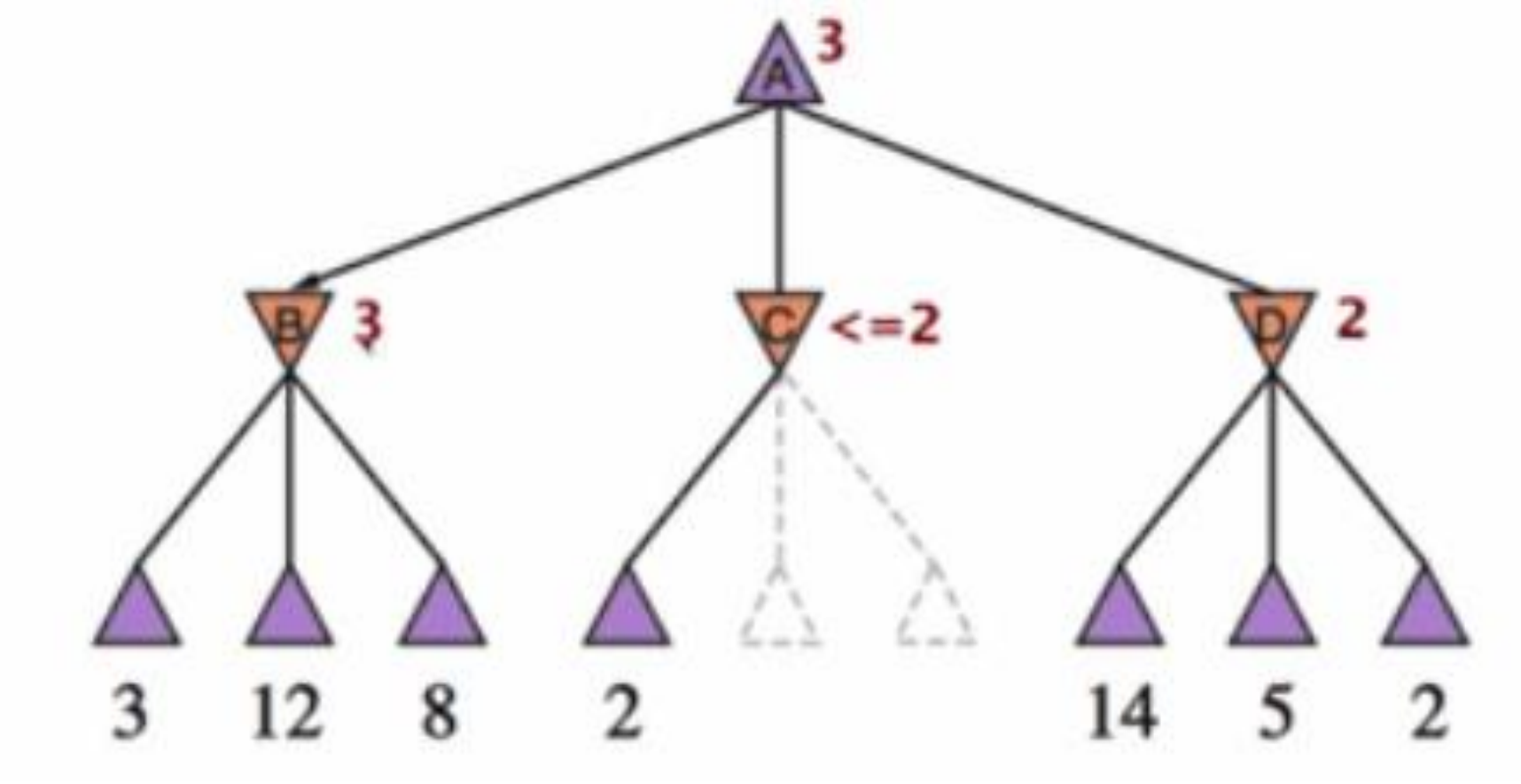
Example: $\alpha - \beta$ pruning



Example: $\alpha - \beta$ pruning

MAX

MIN



$\alpha - \beta$ pruning Algorithm

/* Find the child state with the lowest utility value */

MIN

```
function MINIMIZE(state, α, β)
    returns TUPLE of ⟨STATE, UTILITY⟩ :

    if TERMINAL-TEST(state):
        return ⟨NULL, EVAL(state)⟩

    ⟨minChild, minUtility⟩ = ⟨NULL, ∞⟩

    for child in state.children():
        ⟨_, utility⟩ = MAXIMIZE(child, α, β)

        if utility < minUtility:
            ⟨minChild, minUtility⟩ = ⟨child, utility⟩

        if minUtility ≤ α:
            break

        if minUtility < β:
            β = minUtility

    return ⟨minChild, minUtility⟩
```

Cut

Update β

/* Find the child state with the highest utility value */

MAX

```
function MAXIMIZE(state, α, β)
    returns TUPLE of ⟨STATE, UTILITY⟩ :

    if TERMINAL-TEST(state):
        return ⟨NULL, EVAL(state)⟩

    ⟨maxChild, maxUtility⟩ = ⟨NULL, −∞⟩

    for child in state.children():
        ⟨_, utility⟩ = MINIMIZE(child, α, β)

        if utility > maxUtility:
            ⟨maxChild, maxUtility⟩ = ⟨child, utility⟩

        if maxUtility ≥ β:
            break

        if maxUtility > α:
            α = maxUtility

    return ⟨maxChild, maxUtility⟩
```

Cut

Update α

/* Find the child state with the highest utility value */

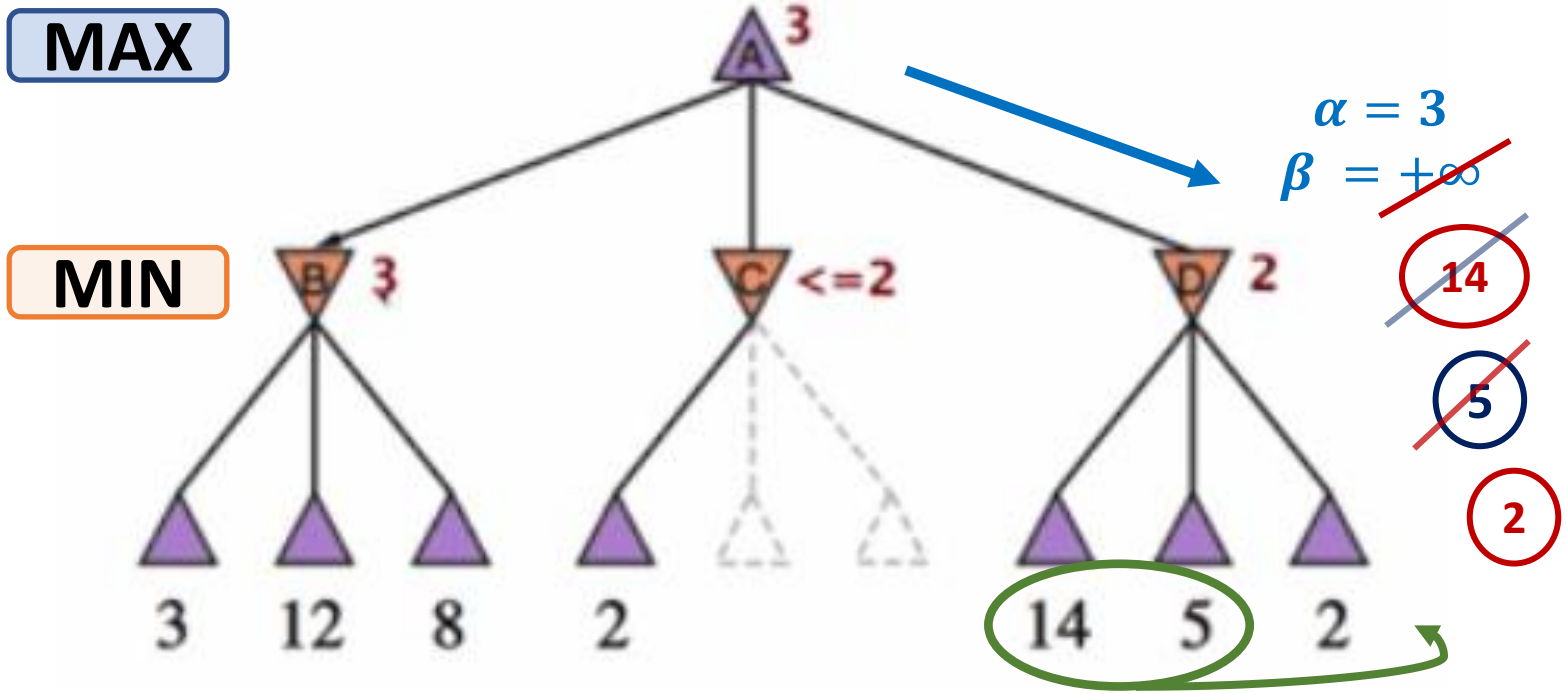
α β

```
function DECISION(state)
    returns STATE :

    ⟨child, _⟩ = MAXIMIZE(state, −∞, ∞)

    return child
```

$$\alpha \geq \beta$$

is the condition to prune (cut) for both maximize and minimize functions

Move Ordering



- It does matter as it affects the effectiveness of $\alpha - \beta$ pruning
- *Example*
  - We could not prune any successor of *D* because the worst successors for Min were generated first
  - If the third one (leaf 2) was generated first we would have pruned the two others (14 and 5)
- *Idea of ordering:* examine first successors that are likely best

$\alpha = 3$
$\beta = +\infty$
2

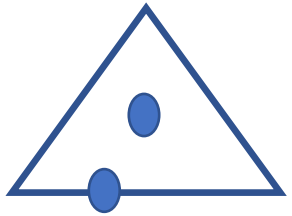
$\alpha \geq \beta$

Move Ordering

- **Worst ordering:** no pruning happens (best moves are on the right of the game tree)
  - **Complexity $O(b^m)$** - which makes the run time of a search with pruning the same as plain Minimax
- **Ideal ordering:** lots of pruning happens (best moves are on the left of the game tree)
  - **Complexity $O(b^{m/2})$** (in practice) - solves tree twice as deep as minimax in the same amount of time
  - The search can go deeper in the game tree
  - In Deep Blue, they found that alpha beta pruning meant the average branching factor at each node about 6 instead of 35
- **How to find good orderings?**
  - Remember the best moves from shallowest nodes
  - Order the nodes so as the best are checked first
  - Use domain knowledge: Need to select "Killer moves", which requires domain knowledge
    - E.g., for chess, try order: *captures* first, then *threats*, then *forward* moves, *backward* moves
  - Bookkeep the states, they may repeat! ("Transposition table")

Real-Time Decisions

- **Minimax:** generates the entire game search space
  - Real games are too large to enumerate tree
- **$\alpha - \beta$ algorithm:** prune large chunks of the trees
- BUT $\alpha - \beta$ still has to go all the way to the leaves of the tree (terminal)
- Impractical in real-time (moves has to be done in a reasonable amount of time)
- ***Solution:*** bound the depth of search (cut off search) and replace ***utiliy(s)*** with ***eval(s)***, an evaluation function to **estimate** value of current board configurations – turns non-terminal nodes into terminal nodes



Real-Time Decisions

- ***eval(s)*** is a heuristic at state ***s***
  - E.g., **Tic-Tac-Toe**:
    h(n) = [# of 3 lengths that are left open for player A] – [# of 3 lengths that are left open for player B]
  - E.g., Alan Turing's function for **Chess**: *h(n) = A(n) - B(n),* where
    - *A(n)* is the sum of the point value for player A's pieces
    - *B(n)* is the sum for player B's piece values
    - Each piece has a material value →

Pawn	1.0
Knight	3.0
Bishop	3.25
Rook	5.0
Queen	9.0

- An ideal evaluation function should rank terminal states in the same way as the utility function, but must be fast
- Typical to define features, make the function a linear weighted sum of the features
- Use domain knowledge to craft the best and useful features

Real-Time Decisions

- Most evaluation functions are specified as a weighted sum of features
- How does it work?
  - Select useful features $\boldsymbol{f_1(s), f_2(s), \ldots, f_n(s)}$
    - Typical evaluation functions use features of the state
    - **Chess:** # of pieces on board (#white pawns, #black pawns, #white queens, #black queens, …)
  - Weighted linear function:

$$eval(s) = \sum_{i=1}^{n} w_i f_i(s)$$

  - Learn w_i from examples
  - Deep blue uses about 6,000 features in its evaluation function!

Pawn	1.0
Knight	3.0
Bishop	3.25
Rook	5.0
Queen	9.0

value of pieces

Stochastic Games

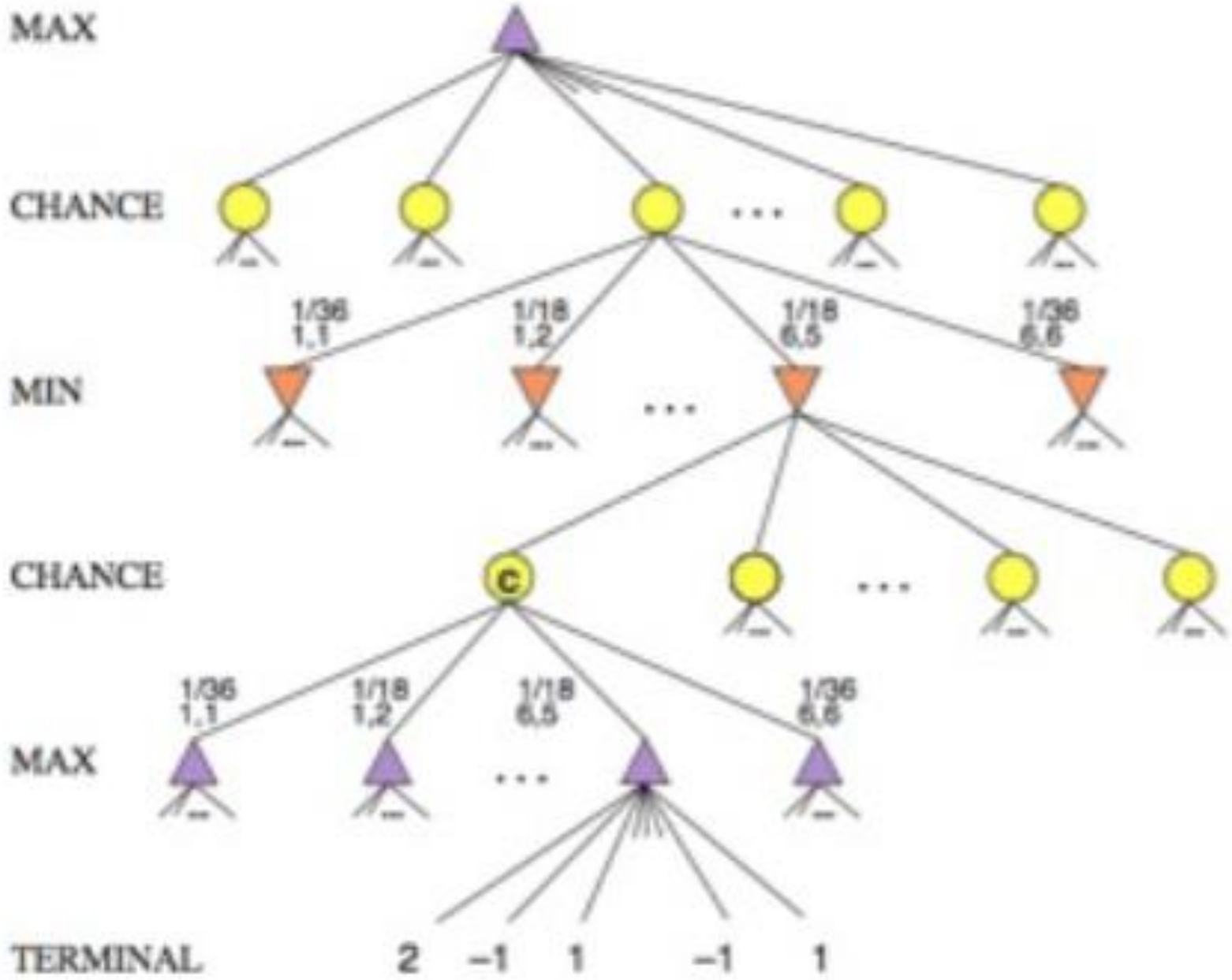
	Deterministic	**Chance** ***(Nondeterministic or Stochastic)***
Perfect Information ***(Fully Observable)***	Chess, Checkers, Go, Othello	Backgammon, Monopoly
Imperfect Information ***(Partially Observable)***	Battleships, Blind Tic-Tac-Toe	Bridge, Poker, Scrabble, Nuclear War

Stochastic Games

- Include a random element (e.g., throwing a dice, shuffling of cards)
- Include **chance nodes**

- **Backgammon:** old board game combining skills and chance
  - The goal is that each player tries to move all of his pieces off the board before his opponent does

Stochastic Games



Partial game tree for Backgammon

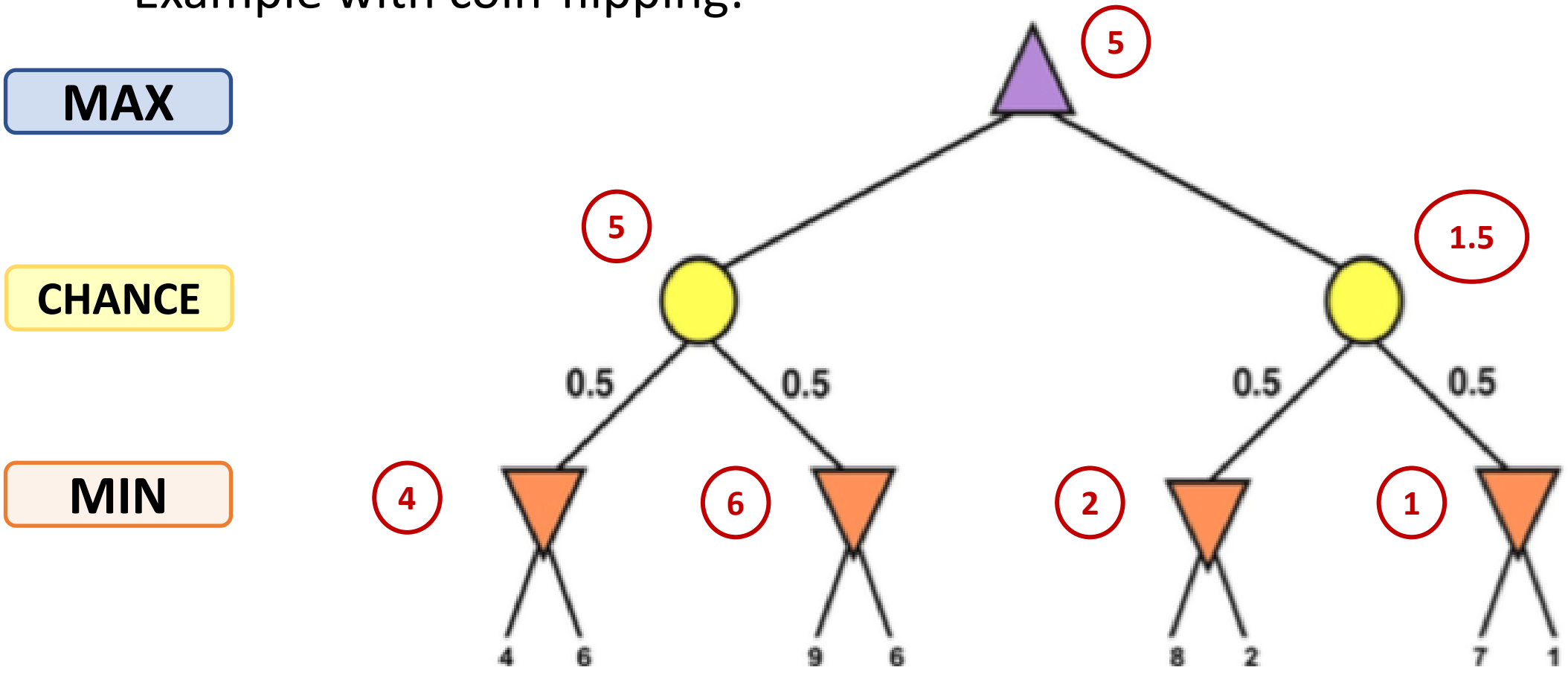
Stochastic Games

Algorithm **Expectiminimax** generalized Minimax to handle chance nodes as follows:

- If state is a **Max** node then
  return the **highest** Expectiminimax-Value of Successors(state)
- If state is a **Min** node then
  return the **lowest** Expectiminimax-Value of Successors(state)
- If state is a **chance** node then
  return **average** of Expectiminimax-Value of Successors(state)

Stochastic Games

Example with coin-flipping:



Expectiminimax

For a state s:

Expectiminimax(s) =

$$\begin{cases} Utility(s) & \text{if Terminal-test(s)} \\ max_{a \in Actions(s)} \text{ Expectiminimax(Result(s,a))} & \text{if Player(s) = Max} \\ min_{a \in Actions(s)} \text{ Expectiminimax(Result(s,a))} & \text{if Player(s) = Min} \\ \sum_r P(r) \text{ Expectiminimax(Result(s,r))} & \text{if Player(s) = Chance} \end{cases}$$

Where ***r*** represents all chance events (e.g., dice roll), and ***Result(s,r)*** is the same state as ***s*** with the result of the chance event is ***r***

Games: Conclusions

- Games are modeled in AI as a search problem and use heuristic to evaluate the game
- Minimax algorithm choses the best move given an optimal play from the opponent
- Minimax goes all the way down the tree which is not practical give game time constraints
- Alpha-Beta pruning can reduce the game tree search which allow to go deeper in the tree within the time constraints
- Pruning, bookkeeping, evaluation heuristics, node re-ordering and IDS are effective in practice

Games: Conclusions

- Games is an exciting and fun topic for AI
- Devising adversarial search agents is challenging because of the huge state space
- We have just scratched the surface of this topic
- Further topics to explore include partially observable games (card games such as bridge, pocker, etc.)
- Except for robot football (a.k.a. soccer), there was no much interest from AI in physical games (see http://www.robocup.org/)